

CALIFORNIA STATE UNIVERSITY, NORTHRIDGE

TCP/NON-TCP-AWARE SERVICE FUNCTION CHAIN (SFC) PLACEMENT IN
NETWORK FUNCTION VIRTUALIZATION

A thesis submitted in partial fulfillment of the requirements
for the degree of Master of Science in
Computer Science

By
Sri Sai Rajesh Venna

December 2025

The thesis of Sri Sai Rajesh Venna is approved:

_____	_____
Marjan Asadinia, PhD.	Date
_____	_____
Robert D McIlhenny, PhD.	Date
_____	_____
Maryam Jalali, PhD., Chair	Date

California State University, Northridge

Acknowledgments

I would like to express my sincere gratitude to all who have supported me in the completion of this thesis. First and foremost, I am deeply indebted to my thesis advisor for their invaluable guidance, constant encouragement, and insightful mentorship throughout this research project. Their expertise and patience were instrumental in shaping this work and my development as a researcher. My sincere thanks are extended to the members of my thesis committee. Their constructive feedback and thoughtful questions challenged me to refine my ideas and significantly improved the quality of this manuscript. I am also grateful to the faculty and staff of the Department of Computer Science at California State University, Northridge, for fostering a stimulating academic environment and for providing the resources necessary for my studies. On a personal level, I owe my deepest gratitude to my family. Their unconditional love, unwavering support, and endless encouragement have been my foundation. This achievement would not have been possible without them. Finally, I wish to thank my friends and colleagues for their camaraderie, support, and the many helpful discussions that have enriched my academic experience.

Table of Contents

Signature Page	ii
Acknowledgments.....	iii
List of Figures	vi
List of Tables	vii
Abstract	viii
CHAPTER 1: INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Research Objectives	2
1.4 Key Contributions	2
1.5 Thesis Outline	3
CHAPTER 2: LITERATURE REVIEW	4
2.1 Introduction.....	4
2.2 Foundations of Service Function Chaining and Traffic Management.....	4
2.3 Algorithm VNF Placement and Routing.....	5
2.4 Comparative Analysis and Research Implications	7
2.5 Research Gap	7
CHAPTER 3: SYSTEM MODEL AND DESIGN	9
3.1. System Architecture.....	9
3.2. Network Model	10
3.3. Traffic Model and Differentiated Service Chains	11
3.4. Formal Problem Formulation.....	12
CHAPTER 4: PROTOCOL-AWARE PLACEMENT ALGORITHMS	17
4.1. Overview of the Algorithmic Strategy.....	17
4.2. Baseline Algorithm: Greedy Placement.....	17
4.3. Advanced Algorithm: Dynamic Programming Placement.....	18

4.4. Optimal Algorithm: Mixed-Integer Linear Programming (MILP)	24
CHAPTER 5: IMPLEMENTATION AND EXPERIMENTAL SETUP.....	31
5.1. Simulation Framework Implementation	31
5.2. Dataset and Traffic Processing.....	32
5.3. Experimental Environment	32
5.4. Performance Evaluation Metrics	33
CHAPTER 6: RESULTS AND DISCUSSION	34
6.1. Comparative Performance: Flow Acceptance and Latency	34
6.2. Analysis of Resource Efficiency and Scalability	35
6.3. Discussion and Summary of Results	37
CHAPTER 7: CONCLUSION AND FUTURE WORK.....	39
7.1. Conclusion	39
7.2. Limitations	40
7.3. Future Work.....	40
REFERENCE	42
APPENDIX	47

List of Figures

Figure 1: The 42-node NSFNet topology used for performance evaluation.	9
Figure 2: Acceptance Ratio (%).....	34
Figure 3: Mean Latency (ms)	35
Figure 4: CPU Utilization (%).....	36
Figure 5: 5-VNF SFC on the 42-node topology using the MILP algorithm	36

List of Tables

Table 1: Acceptance Ratio Across Different SFC Lengths (42-Node NSFNet).....	37
--	----

Abstract

TCP/NON-TCP-AWARE SERVICE FUNCTION CHAIN (SFC) PLACEMENT IN NETWORK FUNCTION VIRTUALIZATION

By

Sri Sai Rajesh Venna

Master of Science in Computer Science

Network Function Virtualization (NFV) promises the greatest agility in deployments of network services, but the potential is often hampered by the problem of managing the heterogeneous network traffic. Traditional Service Function Chain (SFC) placement algorithms are usually "traffic-agnostic" and cannot deal with the conflicting Quality of Service (QoS) requirements of latency-sensitive TCP flows and throughput-sensitive Non-TCP flows. This monolithic approach results in less than optimal resource deployment, poor application performance, and inefficient network operation. This thesis aims to fill this important gap by design, implementation and evaluation of a comprehensive protocol-aware SFC placement framework. The proposed system has the functionality of ingesting and classifying the network traffic which is carried in the real world, and applying the differentiated service chain with different optimization targets for TCP and Non-TCP flows. To solve this placement problem, three algorithms corresponding to three different points on the performance-complexity curve are systematically compared: a fast Greedy heuristic and a path-level optimal Dynamic Programming (DP) approach and a globally optimal Mixed-Integer Linear Programming (MILP) model as a theoretical benchmark. The performance of these algorithms was evaluated through extensive simulations on the 42-node NSFNet topology, with scenarios of varying Service Function Chain lengths from 3 to 8 VNFs. The results show that simplistic heuristics lead to severe resource fragmentation and low acceptance rates of flow despite sometimes low latency. Crucially, the results reveal a large performance gap between the heuristics that scale and the theoretical optimum. This work concludes that a protocol-aware approach is crucial and that the selection of algorithm has drastic implications for network performance underscoring the critical need for advanced, globally-aware heuristics that can effectively balance scalability and near-optimal performance for modern NFV deployments.

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

The architecture of modern networking is undergoing a foundational evolution, shifting from fixed, hardware-based architectures to a more dynamic, software-defined architecture. At the heart of this evolution is Network Function Virtualization (NFV) which separates network functions such as firewalls, load balancers, and routers from proprietary hardware appliances. Instead, these functions are included as software instances known as Virtualized Network Functions (VNFs), on commodity servers [1]. This approach gives network operators unprecedented flexibility, scalability and cost-efficiency for rapidly deploying and managing network services according to dynamic demand.

Service Function Chaining (SFC) is a key enabling feature of NFV. This mechanism enables the construction of composite network service by redirecting the specific traffic through a list of VNFs in an order. An example of this is that a user's web traffic may first pass through a proxy, a firewall and then a Network Address Translation (NAT). The overall performance of the NFV infrastructure heavily depends on efficient placement of these VNFs across the physical network, and on the efficient mapping of traffic between them.

However, the effectiveness of this process is threatened by the natural heterogeneity of modern network traffic. A major difference is the TCP-based traffic (e.g., HTTP, FTP) is reliability-sensitive, very sensitive to latency and packet loss, while Non-TCP traffic (e.g., UDP, RTP for VoIP and video streaming), on the other hand, is less sensitive to minor variations in latency and packet loss, it requires high, consistent throughput [2]. Most conventional SFC placement algorithms are "traffic-agnostic," which employ a single optimization approach for all flows. A single-sided approach results in ineffective resource distribution and deteriorated user experience, which motivates the implementation of the protocol-aware placement strategy that is able to intelligently support these heterogeneous QoS requirements.

1.2 Problem Statement

The systematic problem that has been considered in this study is the strong performance degradation and the inefficiency in resource usage caused by the traffic-agnostic Service Function Chain (SFC) placement in Network Function Virtualization (NFV) environments. By providing equal treatment for all network flows, existing paradigms of network placement don't support the conflicting QoS requirements of varying protocols [3]. This results in scenarios where latency-sensitive TCP traffic flows over poor paths and throughput-based Non-TCP

traffic is slowed down due to unnecessary VNF processing, eventually negating the gain of NFV. The basic problem is the lack of a comprehensive integration, which can dynamically classify flows and apply differentiated placement policies accordingly.

This leads to the primary research question: How can an SFC placement framework be designed to dynamically classify network traffic and apply distinct placement strategies for TCP and Non-TCP flows to simultaneously optimize for low latency and high throughput, respectively, within a unified system?

1.3 Research Objectives

To comprehensively address the problem statement, this thesis aims to achieve the following specific and measurable goals:

- To develop an integrated traffic classification mechanism capable of accurately distinguishing between TCP and Non-TCP flows at the SFC ingress based on protocol signatures and packet headers.
- To design and implement differentiated SFC mapping and placement strategies that tailor the sequence of VNFs and the selection of network paths to the specific QoS requirements of each traffic type, prioritizing low latency for TCP and high throughput for Non-TCP.
- To adapt and compare multiple algorithmic approaches specifically, a Greedy heuristic, a Dynamic Programming (DP) based method, and an optimal Mixed-Integer Linear Programming (MILP) formulation to solve the protocol-aware placement problem.
- To systematically evaluate the performance of the proposed protocol-aware algorithms against one another, using metrics such as end-to-end latency, flow acceptance ratio, and resource utilization within a reproducible simulation environment powered by realistic network traffic data.

1.4 Key Contributions

This thesis presents a novel integrated framework that unifies traffic classification, differentiated VNF placement, and performance evaluation within a single protocol-aware environment:

1. An Integrated, End-to-End Framework: It includes the design and implementation of an integrated framework, which includes the real-life traffic classification,

differentiated VNF placement algorithms, and performance evaluation. This overcomes a major gap where previous studies tended to look at these key elements in an atomised manner.

2. **Systematic Algorithmic Comparison:** The research gives a thorough comparative evaluation of three different placement algorithms (Greedy, Dynamic Programming and MILP) within the identical protocol aware framework. This provides important insights on the performance-complexity trade-offs involved in solving the differentiated SFC placement problem.
3. **A Secure and Appropriate Implementation:** It provides a Python-based simulation environment, which uses real network traffic traces captured from a network monitoring tool. These realistic network conditions ensure that the results are of high practical importance to network operators and Internet Service Providers (ISPs).
4. **Performance Improvements:** The work provides quantitative performance evaluations that demonstrate that a protocol-aware placement approach can provide substantial improvements in important performance metrics. More specifically, it shows improved latency for TCP flows and better resource utilization than the current traffic-agnostic solutions.

1.5 Thesis Outline

This thesis can be summarized as follows. Chapter 2 gives an extensive review of the literature regarding the basics of NFV, SFC and existing placement algorithms while pointing out the gap in research this work fills. In Chapter 3, the proposed system model is formulated, and the network and traffic models are provided as well as the formal problem formulation. Chapter 4 explains the structure and rationale of the three placement algorithms that are implemented. Chapter 5 describes the implementation of the simulation framework, experimental setup and evaluation metrics. Chapter 6 introduces and addresses the empirical findings in the comparative analysis. Finally, Chapter 7 wraps up the thesis and summarizes the findings and suggests possibilities for future research.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

Network Function Virtualization (NFV) has introduced a new level of flexibility, scalability, and operational efficiency enabled by advances in network architecture. In this paradigm shift, proprietary hardware appliances are replaced by software-defined Virtualized Network Functions (VNFs) running on general-purpose computing infrastructure. Service Function Chaining (SFC) is an important part of the NFV, as it enables the dynamic chain of VNFs to serve and handle network traffic. NFV performance is directly related to the optimal placement of VNFs and the intelligent routing of data in service chains since the requirements of different types of traffic are not often homogeneous and can even be mutually exclusive. This heterogeneity is observable in the variations between TCP-based flows and non-TCP-based flows (UDP, IPsec and RTP) which further complicates the optimization of SFC strategies.

2.2 Foundations of Service Function Chaining and Traffic Management

According to Adoga and Pezaros (2022), SFC has become a key enabler of network services today, enabling operators to build tailored processing paths for specific traffic flows. SFC provides unmatched service delivery agility, where traditional, static hardware-based chains are replaced with a dynamic insertion, removal, and reordering of network functions. However, the effectiveness of an SFC depends on the VNF placement strategy. The location of VNFs deployment in the network infrastructure has a direct effect on performance indicators such as latency, throughput, and resource usage (Mahzabeen Emu et al., 2020). Initial SFC placement studies tended to concentrate on the generic optimization goals, which consider all the traffic the same. Yet, the increased popularity of application-specific Quality of Service (QoS)-related needs has further nurtured the field towards traffic-conscious SFC.

Muhammad Usman Sheikh and Peng (2022) stated that traffic awareness is initiated by strong classification at the network ingress. Complex apparatuses are also needed to reliably recognize the protocol and nature of a flow, differentiating, e.g., connection-oriented reliable TCP traffic or connectionless non-TCP traffic that is frequently real-time. This classification is often based on deep packet inspection, port numbers and protocol headers and provides the intelligence required for differentiated treatment. After categorization, the underlying issue is how to make VNF placement and path-selection match the individual needs of different types of traffic (Bi Yu Chen et al., 2020). TCP flows (including their retransmission protocols and delay sensitivity) traditionally require being placed on network nodes and routes with minimal

latency, low jitter, and minimal packet loss. This is frequently linked with giving priority to nodes that have special TCP acceleration facilities or effective congestion control.

Conversely, Non-TCP flows such as those associated with real-time voice, video and interactive gaming tend to be concerned with high throughput and low as well as constant latency. Often, these flows will exploit stateless processing and may bypass network capabilities that introduce unwarranted latency. Liu et al., (2019) argues that the nature of the traffic determines the preferred routes of the candidate, and algorithms usually give preference to the routes with low latency of TCP traffic and the routes with maximum capacity of Non-TCP traffic. This is further complicated by the intrinsic characteristics of VNFs, such as processing overhead, CPU consumption, and stateful versus stateless operation. Placement decision should be put into consideration, therefore, to guarantee operational efficiency and also resource utilization.

2.3 Algorithm VNF Placement and Routing

The placement and routing of Virtual Network Functions (VNFs) inside Service Function Chains (SFCs) is an NP-hard optimization problem which becomes harder with the inclusion of traffic differentiation. Researchers have therefore examined a wide range of algorithmic approaches (from exact mathematical formulations to relatively fast heuristic and meta-heuristic solutions), aiming to achieve a balance between optimality, scalability, and Quality-of-Service (QoS) guarantees.

Mixed-Integer Linear Programming Models (MILP)

Early research in SFC optimization has been based on, numerous, Mixed-Integer Linear Programming (MILP) formulations where the capacities of nodes, bandwidth of links, and service ordering constraints are modeled by linear inequalities. Mostafavi et al., (2021) showed that MILP models can be used to obtain globally optimal placements by minimizing end-to-end latency or maximizing throughput, however their scalability is severely limited because the number of variables grows exponentially with the number of flows, candidate paths and VNFs. Consequently, MILP is now mostly used for evaluating approximation solutions rather than as a practical real-time solution. Sharif et al., (2025) confirmed this limitation where even medium-sized topologies involve a high amount of solver time and thus MILP is not suitable to handle dynamic NFV environments.

Heuristic Methods and Greedy Methods

To overcome the computational cost of MILP, numerous studies propose heuristic or greedy algorithms where placements are constructed incrementally. Verma and Jain (2024) propose a number of resource-aware heuristics that assign each VNF to the first feasible node along candidate paths, while respecting ordering constraints. Such approaches are computationally light and can react fast to changing traffic loads, making them attractive for use with large networks. However, due to their locally optimal decisions from the perspective of the future placements, they often suffer from resource fragmentation and reduced flow acceptance ratios under high load. However, there's a point here for Sharif et al., (2025), that heuristics succeed in scalability at the cost of a substantial deviation to the global optimum determined by MILP solutions.

Dynamic-Programming (DP) Techniques

Dynamic Programming techniques address the sequential dependencies among VNFs in a chain by breaking the whole problem into smaller parts. Verma and Jain (2024) and related research are used to modify the DP to calculate the minimum cumulative cost for the first i VNFs placement of a chain of VNFs on m nodes on a path. This approach provides the path-level optimality because all the intermediate placements are evaluated in a way that it explores, but the complexity becomes prohibitive with $O(k * m^2)$ where k is the number of VNFs and m is the number of nodes in the path. DP therefore works well for optimizing moderate-sized paths or single-flow optimization but it becomes inefficient for large-scale or multi-flow computations. Unlike greedy heuristics, DP can take multiple QoS measures (latency, jitter, bandwidth) into its recurrence relation which makes it especially appropriate for differentiated SFCs.

Meta-heuristic Strategies and Hybrid Strategies

Recent researches trends are a combination of the efficiency of heuristics and the power of exploration of meta-heuristics such as Genetic Algorithms (GA), Particle Swarm Optimization (PSO), or Ant Colony Optimization (ACO). Taha et al., (2025) review GA-based frameworks which evolve candidate placements using crossover and mutation operators, which can find near-optimal solutions within limited computation time. These approaches deal with multiple, conflicting objectives, such as e.g. minimize latency while balancing load across nodes. Hybrid models are also found in recent publications: In a new hybrid GA - MILP method, Sharif et al., (2025) suggested that MILP can be used to supply the cost evaluation function for the GA population, which improves the convergence and the quality of solutions. Despite these facts,

meta-heuristics still have to be tuned carefully with regard to their parameters and can give inconsistent results in different runs.

2.4 Comparative Analysis and Research Implications

Collectively, the literature identifies a clear trading relationship between optimality and scalability. MILP models are theoretically good upper bounds on performance but are computationally infeasible for large dynamic networks. Heuristics are efficient in terms of scale and are tend to produce sub-optimal or resource-fragmented allocation outcomes. Dynamic-Programming approaches provide an intermediate solution achieving per-path optimality at moderate computational cost while meta-heuristics offer flexible, near-optimal results with manageable run time, if properly tuned. Most of the studies found, though, are based on homogeneous traffic and do not explicitly distinguish between TCP and Non-TCP flows when defining optimization objectives. This gap emphasizes the need for an integrated and protocol-aware framework that compares multiple algorithmic strategies in a systematic manner under heterogeneous QoS requirements - an objective addressed in this thesis.

2.5 Research Gap

Although a huge amount of research has been conducted regarding Service Function Chain (SFC) placement and routing, most existing solutions address the problem in an isolated manner. Prior work has typically tried to focus on either (a) developing optimization models such as MILP for small-scale networks, (b) scalable heuristics and meta-heuristics for large deployments, or (c) simulation environment for the evaluation of placement strategies. However, such studies rarely combine the importance of traffic classification, algorithmic optimization and empirical evaluation in one end-to-end framework. Furthermore, most of the previous attempts assume uniform network traffic and do not account for the heterogeneous Quality-of-Service (QoS) requirements of TCP and Non-TCP flows. As a result, existing solutions cannot effectively balance latency-sensitive and throughput-sensitive demands in the same NFV infrastructure.

Another important gap in the literature is the lack of systematic evaluation of different algorithmic paradigms (i.e. exact (MILP), path-level (Dynamic Programming) and heuristic or meta-heuristic (Greedy/Genetic) methods) under unified experimental conditions. Without such comparison, the trade-offs between computational complexity, scalability and optimality are not well understood.

To cover these gaps, a protocol-aware SFC placement framework based on real-world traffic classification, differentiated VNF mapping strategies, and comparative study of three algorithmic strategies is proposed in the present research. By combining scalability and near-optimal performance, the study provides a unified and practical solution for heterogeneous NFV environments.

CHAPTER 3: SYSTEM MODEL AND DESIGN

3.1. System Architecture

The proposed protocol-aware SFC placement framework is built as a cohesive, end-to-end system to logically map high level network policy and resource allocation. Its architecture is also modular so that there is a clear separation in concerns between the first steps of traffic analysis and the final steps of VNF deployment. The information flows starting from the network edge and passing through various specialized elements, as shown in the system architecture diagram.

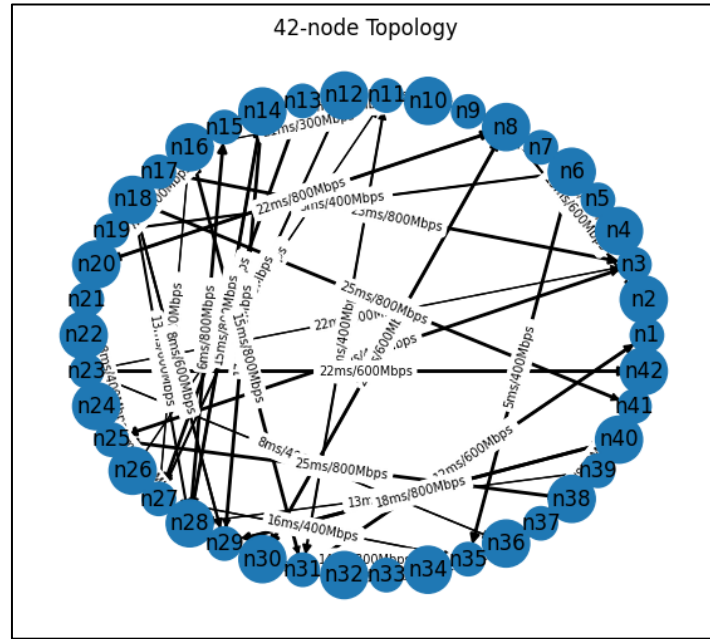


Figure 1: The 42-node NSFNet topology used for performance evaluation.

The framework consists of four main parts:

1. **Traffic Classifier:** It is the entry point of the system and is tasked with the task of examining incoming network flow. By detecting packet headers and protocol signatures, it is able to classify each flow as TCP or Non-TCP. This base classification is the all-important trigger for all further differentiated processing and is the basis for the framework's protocol-aware intelligence.
2. **SFC Orchestrator:** This unit acts as a central policy and control unit, which receives the flow type from the classifier. Based on predefined rules it determines the respective SFC a more complex SFC for TCP or a simpler one for Non-TCP [7]. It then encapsulates the flow information and as well how to create the VNF sequence into a placement request that is sent to the optimization engine.

3. **Placement Algorithm Module:** This is the core computational engine of the framework. It takes in the placement request coming from the orchestrator and applies a specific algorithm (Greedy, Dynamic Programming or MILP) to solve the complex optimization problem [8]. It is solely responsible for selecting the most appropriate physical nodes within the substrate network to deploy all the VNFs within the chain, in a manner that adheres to all the resource constraints and optimizes with respect to the specific QoS goals of the flow.
4. **Substrate Network (NFVI):** This corresponds to the physical infrastructure, which is a collection of servers (nodes) with limited computing resources with each other linked through network connections with certain bandwidth and latency properties. The placement module communicates with a model of the network for making decisions.

3.2. Network Model

The underlying NFV infrastructure is modeled as a directed graph

$$G = (V, E)$$

where

$$V = \{v_1, v_2, \dots, v_n\}$$

represents the set of physical substrate nodes, and

$$E = \{e_{ij} \mid v_i, v_j \in V\}$$

represents the set of directed links connecting these nodes.

Each substrate node $v_i \in V$ corresponds to a physical server or switch capable of hosting one or more Virtual Network Functions (VNFs). Every node is characterized by a set of resource attributes, including CPU capacity C_i (in GHz), available memory M_i , and a binary indicator A_i denoting the presence of TCP-acceleration capability. Similarly, each directed link $e_{ij} \in E$ is associated with bandwidth B_{ij} (in Gbps) and transmission latency L_{ij} (in ms). These parameters collectively determine the feasibility of deploying a given service chain along a candidate path.

The physical network topology adopted in this study is the NSFNet topology, which consists of 42 nodes and 84 bidirectional links [4]. The topology is implemented using the NetworkX library in Python to maintain explicit node and edge attributes for CPU, bandwidth, and latency. Each simulation run begins with randomly assigned resource capacities drawn from realistic ranges, ensuring heterogeneity in the underlying infrastructure.

To support differentiated SFC deployment, the network model maintains dynamic state variables that record the remaining CPU, memory, and link bandwidth after each flow placement. This enables consistent enforcement of capacity constraints across all placement algorithms described in Chapter 4.

3.3. Traffic Model and Differentiated Service Chains

The defining element of service delivery is a network flow, which is formally defined as f consisting of a set of identifying parameters. As used in the implementation, these are integrated as:

$$f = \{id, src, dst, protocol, bw_{demand}\}$$

Where:

- id is a unique identifier.
- src and dst are the source and destination IP addresses.
- $protocol$ is the flow classified type ('TCP' or 'Non-TCP').
- bw_{demand} is a flow's bandwidth requirement in Mbps.

Applying the classified protocol, a differentiated Service Function Chain (SFC) has been developed as one of the key technologies and is adopted in this research. Furthermore, it ensures that flows pass through VNFs that are relevant to their QoS needs by optimizing performance and minimizing resource consumption.

The two distinct SFCs are defined as follows:

1. TCP Service Chain:

Proxy $\rightarrow$ Firewall $\rightarrow$ NAT

This chain is for connection-oriented, delay and reliability-sensitive TCP streams. It contains a stateful Proxy VNF (e.g., deep packet inspection, caching, or security filtering functions) that are particularly relevant for TCP-based applications such as HTTP.

2. Non-TCP Service Chain:

Firewall $\rightarrow$ NAT

This lighter chain profile is designed for Non-TCP flows (including UDP-based streaming, VoIP, etc.). It intelligently omits the Proxy VNF, since connectionless and real-time traffic handled in a stateful manner would otherwise lead to increased latency and unnecessary CPU utilization without delivering any additional benefit. This

distinction is one of the most important sources of efficiency improvement in the proposed system.

Each VNF in these service chains has an associated resource cost (CPU per Mbps of traffic), which is considered in defining the placement constraints of the system.

3.4. Formal Problem Formulation

The protocol-aware SFC placement problem can be regarded as finding an optimal allocation of the VNFs of all incoming flows into the nodes and paths of the substrate network, while satisfying the resource constraints and protocol-specific QoS requirements (latency for TCP flows and throughput for Non-TCP flows).

3.4.1 Set and Parameters

- Substrate network:

$$G = (N, E)$$

where,

- N is the set of physical nodes,
- E is the set of directed links.
- Node capacities:
For each node $n \in N$:
 - C_n^{cpu} = CPU capacity (e.g., in GHz).
- Link capacities:
For each link $(u, v) \in E$:
 - B_{uv} = available bandwidth (e.g., in Mbps or Gbps),
 - L_{uv} = propagation latency (ms).
- Flow set:
 F = set of incoming flows. For each flow $f \in F$:
 - s_f = source node,
 - d_f = destination node,
 - b_f = required bandwidth of the flow,
 - $\tau_f \in \{\text{TCP}, \text{NonTCP}\}$ = protocol type of the flow,

- $\mathcal{S}_f = (v_{f,1}, v_{f,2}, \dots, v_{f,K_f})$ = ordered Service Function Chain (SFC) for flow f ,
where $v_{f,i}$ is the i -th VNF in the chain.
- VNF resource parameters:
For each flow f and position $i = 1, \dots, K_f$:
 - $\alpha_{f,i}$ = CPU requirement per Mbps for VNF $v_{f,i}$.
The total CPU load of this VNF for flow f is

$$\alpha_{f,i} \cdot b_f.$$

- Candidate paths:
For each flow f , let $\mathcal{P}_f$ be the set of pre-computed candidate paths between s_f and d_f .
Each path $p \in \mathcal{P}_f$ is an ordered node sequence:

$$p = (n_1, n_2, \dots, n_{|p|}).$$

For each candidate path $p \in \mathcal{P}_f$, define:

- **Path latency**

$$L_{f,p} = \sum_{(u,v) \in p} L_{uv}$$

- **Inverse-bandwidth cost** (used to favour high-bandwidth paths)

$$\Theta_{f,p} = \sum_{(u,v) \in p} \frac{1}{B_{uv}}.$$

(These path-level quantities are computed once from the network model.)

3.4.2 Decision Variables

- $y_f \in \{0,1\}$
= 1 if flow f is accepted (successfully placed), 0 otherwise.
- $w_{f,p} \in \{0,1\}$ for each flow $f \in F$ and path $p \in \mathcal{P}_f$
= 1 if flow f uses candidate path p , 0 otherwise.
- $z_{f,i,n} \in \{0,1\}$ for each flow $f \in F$, each VNF position $i = 1, \dots, K_f$, and each node $n \in N$
= 1 if the i -th VNF in the chain of flow f is placed on node n , 0 otherwise.

These variables are later instantiated in the MILP model in Chapter 4, but here they are introduced conceptually.

3.4.3 Objective Functions

Because TCP and Non-TCP flows have conflicting QoS needs, the optimization objective is protocol-aware.

- Let $F_{\text{TCP}} = \{f \in F \mid \tau_f = \text{TCP}\}$
 - Let $F_{\text{NonTCP}} = \{f \in F \mid \tau_f = \text{NonTCP}\}$
1. TCP flows - minimise end-to-end latency

For TCP flows, the objective is to minimise the total path latency over all accepted TCP flows:

$$\min \sum_{f \in F_{\text{TCP}}} \sum_{p \in \mathcal{P}_f} L_{f,p} w_{f,p}.$$

2. Non-TCP flows - maximise throughput (equivalently minimise inverse bandwidth)

To favour high-bandwidth paths, we minimise the aggregate inverse-bandwidth cost for accepted Non-TCP flows:

$$\min \sum_{f \in F_{\text{NonTCP}}} \sum_{p \in \mathcal{P}_f} \Theta_{f,p} w_{f,p}.$$

In this thesis, these two goals are combined into a single composite objective of the form:

$$\min \left(\sum_{f \in F_{\text{TCP}}} \sum_{p \in \mathcal{P}_f} L_{f,p} w_{f,p} + \sum_{f \in F_{\text{NonTCP}}} \sum_{p \in \mathcal{P}_f} \Theta_{f,p} w_{f,p} \right),$$

with the understanding that the path costs $L_{f,p}$ and $\Theta_{f,p}$ are chosen to reflect the different priorities of TCP and Non-TCP traffic. (The exact weighting is refined in the MILP model in Chapter 4.)

3.4.4 Constraints

1. **Flow acceptance and path selection**

Each accepted flow is assigned to at most one candidate path:

$$\sum_{p \in \mathcal{P}_f} w_{f,p} = y_f, \quad \forall f \in F.$$

If $y_f = 0$, the flow is rejected and $w_{f,p} = 0$ for all paths p .

2. VNF placement per flow

For each accepted flow and each VNF in its chain, the VNF must be placed on exactly one node:

$$\sum_{n \in N} z_{f,i,n} = y_f, \quad \forall f \in F, i = 1, \dots, K_f.$$

3. CPU capacity constraint

For every physical node, the total CPU of all VNFs placed on it must not exceed its capacity:

$$\sum_{f \in F} \sum_{i=1}^{K_f} \alpha_{f,i} b_f z_{f,i,n} \leq C_n^{\text{cpu}}, \quad \forall n \in N.$$

4. Link bandwidth constraint

For each link, the aggregate bandwidth of all flows routed over that link must not exceed the link capacity. Using the candidate paths, this can be written as:

$$\sum_{f \in F} b_f \sum_{p \in \mathcal{P}_f: (u,v) \in p} w_{f,p} \leq B_{uv}, \quad \forall (u,v) \in E.$$

5. Path-placement consistency constraint

A VNF can only be placed on a node that lies on the chosen path of that flow. For each flow, VNF position and node:

$$z_{f,i,n} \leq \sum_{p \in \mathcal{P}_f: n \in p} w_{f,p}, \quad \forall f \in F, i = 1, \dots, K_f, n \in N.$$

If node n does not belong to any selected path for flow f , then the right-hand side is 0 and the constraint forces $z_{f,i,n} = 0$.

6. Service Function Chain (SFC) ordering constraint

For each flow f , the VNFs in its chain must appear along the selected path in the correct order. Informally, if VNF $v_{f,i}$ is placed on node n and VNF $v_{f,i+1}$ is placed on node n' on the same path, then node n' must appear after node n in the path sequence. This SFC ordering requirement is enforced when constructing candidate paths and is made explicit in the MILP formulation in Chapter 4.

CHAPTER 4: PROTOCOL-AWARE PLACEMENT ALGORITHMS

4.1. Overview of the Algorithmic Strategy

To solve the complex protocol-aware SFC placement problem, this research implements and evaluates a spectrum of 3 different algorithms. Each of the algorithms was selected to represent a different point in the trade-off curve between computational complexity and solution optimality. The first is a fast greedy heuristic, which is used as a performance baseline. The second one is a more sophisticated Dynamic Programming (DP) approach, which is aimed to find a best solution for one flow on the given path. Finally, a Mixed-Integer Linear Programming (MILP) model is constituted that gives a globally optimal solution and it can be seen as a theoretical benchmark for the other methods.

4.2. Baseline Algorithm: Greedy Placement

The Greedy placement algorithm is a solution for the placement problem with low complexity and heuristic based approach, with a focus on speed of execution. It processes flows sequentially, making locally optimal decisions at each step without backtracking and taking into account the global impact on the future flow placements. This approach serves as a practical baseline for evaluating more advanced algorithms.

Step-by-Step Logic:

The algorithm cycles through every flow and tries to uncover the first feasible placement out of a list of pre-computed candidate paths. For every flow, it explores each candidate path. On a path, it tries to position the necessary VNFs according to their position on the path. For every VNF in this chain, it goes through the nodes in the current path in the order of processing. The first node is selected for placement that has enough free CPU capacity. A key protocol-aware capability is introduced here: if the flow is of type TCP, nodes with TCP acceleration capabilities are given priority in this scan. If there exists any VNF in the service chain that cannot be placed on any of the nodes along the path, then the entire path is said to be infeasible and the algorithm is moved to the next candidate path. The first path for which a complete and feasible placement is found is immediately chosen for the flow and the corresponding CPU resources are reduced on the nodes and so on to the next flow considered by the algorithm.

Computational Complexity and Performance:

The complexity of Greedy algorithm is moderate and, therefore, very much scalable. The runtime of the service chain was mostly a function of the number of flows, the number of candidate paths, again the paths themselves, and the length of the SFCs. Its major drawback is that it is very short-sighted. And by making locally optimal decisions, and committing resources forward, it becomes resource restrictive and less than globally optimal. This can result in lower flow acceptance rates on the whole, as early flows can consume critical resources on suboptimal paths such that there will be no options available later.

Algorithm 1: Greedy Placement Pseudocode

Input: A set of flows F , Network Graph G

For each flow f in F :

$is_placed \leftarrow false$

For each candidate path p for f :

If all VNFs in $f.sfc$ can be feasibly placed on nodes in p :

Place the VNFs and reserve resources in G

$is_placed \leftarrow true$

break // Move to the next flow

If is_placed is false:

Reject flow f

4.3. Advanced Algorithm: Dynamic Programming Placement

Dynamic Programming (DP) is used to determine the optimal placement of the VNFs along a chosen candidate path for each flow. While Greedy makes a placement decision only with local information, DP explores all feasible placements sequentially, ensuring protocol-aware optimality with significantly reduced computation compared to a full MILP.

DP breaks the problem into smaller subproblems by considering:

- the first i VNFs in the chain
- placed along the first j nodes of the candidate path

This structure captures the fundamental dependency in SFC placement where the order of VNFs must strictly follow the order of nodes on the path.

4.3.1 DP State Definition

Let the candidate path consist of nodes:

$$p = (n_1, n_2, \dots, n_m)$$

Let the SFC for a flow be:

$$S = (v_1, v_2, \dots, v_K)$$

Define the DP state:

$$DP[i][j] = \text{minimum cost of placing the first } i \text{ VNFs ending at node } n_j$$

Where:

- $i \in \{1, 2, \dots, K\}$ (VNF index)
- $j \in \{1, 2, \dots, m\}$ (path node index)

The cost depends on protocol:

- TCP flows: minimise latency

Non-TCP flows: minimise inverse bandwidth penalty

4.3.2 Initialization

The first VNF v_1 can be placed at node n_j only if that node has sufficient CPU.

$$DP[1][j] = \begin{cases} \text{ProcCost}(v_1, n_j), & \text{if node supports } v_1 \\ \infty, & \text{otherwise} \end{cases}$$

Where:

$$\text{ProcCost}(v_i, n_j) = \text{processing delay at node } n_j$$

4.3.3 DP Transition

To place VNF v_i on node n_j , DP considers all valid previous nodes:

$$DP[i][j] = \min_{k < j} [DP[i-1][k] + \text{LinkCost}(n_k, n_j) + \text{ProcCost}(v_i, n_j)]$$

LinkCost differs by protocol:

- TCP:

$$\text{LinkCost}(n_k, n_j) = L_{k,j} \quad (\text{latency})$$

- Non-TCP:

$$\text{LinkCost}(n_k, n_j) = \frac{1}{B_{k,j}} \quad (\text{inverse bandwidth})$$

This ensures SFC order is preserved because DP only transitions from earlier path nodes $k < j$.

4.3.4 Final Answer

The optimal placement cost for the entire SFC is:

$$\min_{j=1}^m DP[K][j]$$

Backtracking from this state yields the optimal mapping of each VNF to a specific node along the path.

4.3.5 Simple Example (Illustrative)

Consider a flow with:

- **SFC:** $S = (v_1, v_2)$
- **Candidate path:** $p = (A, B, C)$
- **Propagation delays:**
 - $A \rightarrow B = 3 \text{ ms}$
 - $B \rightarrow C = 2 \text{ ms}$
- **Processing delays:**
 - v_1 : $A=2 \text{ ms}$, $B=1 \text{ ms}$, $C=3 \text{ ms}$
 - v_2 : $A=1 \text{ ms}$, $B=2 \text{ ms}$, $C=1 \text{ ms}$
- **Flow type:** TCP (latency minimization)

Step 1: DP Initialization

$$DP[1][A] = 2, \quad DP[1][B] = 1, \quad DP[1][C] = 3$$

Step 2: Computing DP for second VNF

$$DP[2][A] = \infty \quad (\text{no previous nodes})$$

$$DP[2][B] = DP[1][A] + 3 + 2 = 2 + 3 + 2 = 7$$

$$DP[2][C] = \min(DP[1][A] + 3 + 1, DP[1][B] + 2 + 1)$$

$$= \min(6, 4) = 4$$

State	A	B	C
DP[1][·]	2	1	3
DP[2][·]	∞	7	4

Result

Minimum total latency:

$$\min(DP[2][A], DP[2][B], DP[2][C]) = 4$$

Optimal placement:

- v_1 on **B**
- v_2 on **C**

DP thus finds the optimal VNF placement along the path for the chosen protocol.

DP Placement:

```
function DP_Place_On_Path(path, sfc, G, cpu_state, flow_bw)

    m ← len(path)          // number of nodes on path

    k ← len(sfc)           // number of VNFs in SFC

    INF ← a very large number

    // DP and bookkeeping arrays

    DP[0..k-1][0..m-1] ← INF

    Prev[0..k-1][0..m-1] ← -1    // store previous node index

    Snap[0..k-1][0..m-1] ← NULL // CPU snapshot after placement

    // Base case: place first VNF v0 at node j
```

```

for j in 0 .. m-1:

    node ← path[j]

    vname ← sfc[0]

    cpu_need ← cpu_req(vname, flow_bw) // e.g. cpu_per_mbps * flow_bw

    if cpu_state[node] ≥ cpu_need:

        // latency from flow source mapped to path start up to node j

        lat_src_to_j ← latency_along_segment(path, 0, j)

        DP[0][j] ← lat_src_to_j + proc_delay(vname)

        Snap[0][j] ← copy(cpu_state); Snap[0][j][node] -= cpu_need

    else:

        DP[0][j] ← INF

// Recurrence: place i-th VNF at node j (j must be ≥ i)

for i in 1 .. k-1:

    vname ← sfc[i]

    cpu_need_i ← cpu_req(vname, flow_bw)

    for j in i .. m-1: // ensure ordering (at least i nodes)

        node_j ← path[j]

        // try all valid previous placements t where t < j

        for t in (i-1) .. (j-1):

            if DP[i-1][t] == INF: continue

            prev_snap ← Snap[i-1][t]

            if prev_snap is NULL: continue

            if prev_snap[node_j] < cpu_need_i: continue

```

```

    // latency between node at index t and node at index j
    seg_lat ← latency_along_segment(path, t, j)

    cand_cost ← DP[i-1][t] + seg_lat + proc_delay(vname)

    if cand_cost < DP[i][j]:
        DP[i][j] ← cand_cost

        Prev[i][j] ← t

        new_snap ← prev_snap.copy(); new_snap[node_j] -= cpu_need_i

        Snap[i][j] ← new_snap

// Find best end position and include tail to destination
best_cost ← INF; best_j ← -1

for j in (k-1) .. (m-1):
    if DP[k-1][j] == INF: continue

    tail_lat ← latency_along_segment(path, j, m-1) // from last VNF node to path end (dst)

    total ← DP[k-1][j] + tail_lat

    if total < best_cost:
        best_cost ← total; best_j ← j

if best_j == -1:
    return (NULL, NULL) // infeasible on this path

// Backtrack to construct assignment
assignment ← empty map

i ← k-1; j ← best_j

```

```

while i ≥ 0:

    assignment[i] ← path[j]

    j ← Prev[i][j]

    i ← i - 1

return (assignment, best_cost)

end function

```

4.4. Optimal Algorithm: Mixed-Integer Linear Programming (MILP)

The MILP formulation gives a way of finding the globally optimal solution for a whole set of flows at the same time. It is an invaluable academic benchmark to be used to measure the “optimality gap” of the heuristic and DP-based approaches.

Complete MILP Model:

Decision Variables:

- $y_f \in \{0,1\}$: Binary variable representing whether flow f is accepted (1) or rejected (0).
- $w_{f,p} \in \{0,1\}$: Binary variable indicating whether flow f is placed on candidate path p (1) or not (0).
- $z_{f,i,n} \in \{0,1\}$: Binary variable representing whether the i^{th} VNF of the chain of flow f is assigned to node n (1) or not (0).

Objective Function:

The goal is to minimize a composite cost function which balances latency, throughput prioritization, and flow acceptance.

$$\begin{aligned}
\text{Minimize: } & \sum_{f \in \text{TCP}} \sum_p w_{f,p} \times \text{path_latency}(p) + \sum_{f \in \text{NonTCP}} \sum_p w_{f,p} \times \text{path_cost}(p) \\
& + \sum_f \text{big_penalty} \times (1 - y_f)
\end{aligned}$$

Where $\text{path_cost}(p)$ is a function of the inverse bandwidth, and big_penalty is a large constant to ensure that the solver prioritizes accepting flows.

Constraints:

1. **Flow Acceptance Constraint:** A flow can only be assigned to a path if it is accepted.

For each flow f :

$$\sum_p w_{f,p} = y_f$$

2. **Path Uniqueness Constraint:** Each accepted flow must be assigned to exactly one path (implicit in the constraint above).
3. **VNF Placement Constraint:** For each accepted flow f and each VNF i in its chain, it must be placed on exactly one node n :

$$\sum_n z_{f,i,n} = y_f$$

4. **Placement Binding Constraint:** A VNF can only be placed on a node if that node is part of the path chosen for the flow. For each flow f , VNF i , node n , and path p :

$$z_{f,i,n} \leq \sum_{p \text{ where } n \in p} w_{f,p}$$

5. **CPU Capacity Constraint:** For each node n , the total CPU consumed by all VNFs placed on it cannot exceed its capacity:

$$\sum_f \sum_i z_{f,i,n} \times \text{cpu_cost}(\text{vnf}_i, f) \leq \text{CPU_capacity}(n)$$

6. **Link Bandwidth Constraint:** For each link (u,v) , the total bandwidth of all flows routed over it cannot exceed its capacity:

$$\sum_f \sum_p w_{f,p} \times \text{flow_bandwidth}(f) \times \text{is_link_on_path}(u,v,p) \leq \text{Bandwidth}(u,v)$$

Algorithm 3: MILP Placement (global model)

Inputs

- $G = (V, E)$: substrate graph with $\text{cpu_capacity}[n]$, $\text{bw}[e]$, $\text{latency}[e]$
- Flows : list of flows f with $\text{flow_bw}[f]$, $\text{src}[f]$, $\text{dst}[f]$, $\text{sfc}[f]$ (ordered VNFs)
- $\text{Paths}[f]$: precomputed candidate paths for each flow (each p is an ordered node list)
- vnfs : dictionary for each VNF name v with $\text{cpu_per_mbps}[v]$, $\text{proc_delay}[v]$
- time_limit_s : solver time limit (optional)

Outputs

- $assignment[f]: \{i \rightarrow node\}$ for each accepted flow f (or NULL if rejected)
- $accepted_flows$: list / indicator $y[f]$
- solver statistics: (status, objective, runtime)

MILP Placement:

function BUILD_AND_SOLVE_MILP(G, Flows, Paths, vnfs, time_limit_s)

// 1. CREATE MILP MODEL

Initialize MILP problem P (minimize ...)

// 2. DECISION VARIABLES

For each flow f :

$y[f] \in \{0,1\}$ // 1 if flow f is accepted

For each candidate path p in $Paths[f]$:

$w[f,p] \in \{0,1\}$ // 1 if flow f uses path p

For each VNF index i in $0..k_f-1$ and for each node n in V :

$z[f,i,n] \in \{0,1\}$ // 1 if i -th VNF of flow f placed at node n

For each node n and each VNF type v :

$x[n,v] \in \{0,1\}$ // 1 if VNF type v is instantiated at n

// Optional continuous slack variables to allow soft feasibility

For each node n : $slack_cpu[n] \geq 0$

For each link e : $slack_link[e] \geq 0$

// 3. OBJECTIVE FUNCTION (example composite)

$Obj \leftarrow \sum_f \sum_p (latency(p) * w[f,p])$ // latency cost

$+ \text{BigReject} * \sum_f (1 - y[f])$ // penalize rejections
 $+ \text{SmallInst} * \sum_{n,v} x[n,v]$ // instantiation cost
 $+ \text{CPUSlackPen} * \sum_n \text{slack_cpu}[n]$
 $+ \text{LinkSlackPen} * \sum_e \text{slack_link}[e]$

Set objective Minimize Obj

// 4. CONSTRAINTS

// 4.1 path-choice consistency

For each f: $\sum_p w[f,p] == y[f]$

// 4.2 VNF placement if accepted

For each f and each VNF index i:

$\sum_n z[f,i,n] == y[f]$

// 4.3 binding: placing a VNF implies instantiation $x[n,v]$

For each f,i,n:

$z[f,i,n] \leq x[n, v_name = \text{sfc}[f][i]]$

// 4.4 node CPU capacity (with slack)

For each node n:

$\sum_f \sum_i \text{cpu_per_mbps}[\text{sfc}[f][i]] * \text{flow_bw}[f] * z[f,i,n] \leq \text{cpu_capacity}[n] + \text{slack_cpu}[n]$

// 4.5 link bandwidth capacity (directional)

For each link (u,v):

$\sum_f \sum_p: (p \text{ uses edge } (u,v)) \text{ flow_bw}[f] * w[f,p] \leq \text{bw}[(u,v)] + \text{slack_link}[(u,v)]$

// 4.6 placement on chosen path only

For each f,i,n:

For all p in Paths[f] where node n not in p:

// If flow uses path p, z[f,i,n] must be 0

$z[f,i,n] \leq \sum_{p \in \text{PathsWithN}} w[f,p]$

// (equivalently enforce $z[f,i,n] \leq \sum_p I(n \in p) * w[f,p]$)

// 4.7 ordering constraint (enforce increasing path indices)

// (Implicit by node positions on path combined with constraint 42; explain in text)

// 5. SOLVER SETTINGS & INVOKE SOLVER

Choose MILP solver (e.g., CBC, Gurobi). Set time limit = time_limit_s

Solve P

// 6. EXTRACT SOLUTION

For each f:

$y_val \leftarrow \text{round}(\text{value}(y[f]))$

if y_val == 1:

Select p* with $w[f,p^*] == 1$

For each i:

Find node n with $z[f,i,n] == 1$

assignment[f][i] ← n

else:

assignment[f] $\leftarrow$ NULL

Return (assignment, y, x, solver_stats)

end function

Ordering constraint: enforce VNF order by the path node indices: when you extract assignment[f] ensure the node indices are strictly increasing. Alternatively, the DP step per-path is used to find the best ordering - for MILP the $I(n \in p)$ plus z placement suffices if you require the chosen nodes appear in chain order along the chosen path.

Slack variables: include slack variables with penalties in the objective to keep the MILP solvable even when full feasibility is difficult; report slack totals in solver output.

Solver / runtime:

Specify the solver used (e.g., CBC for reproducibility or Gurobi if licensed) and the time limit. Add a one-line complexity note: MILP is NP-hard worst-case runtime grows exponentially; runtime depends on number of flows $\times$ candidate paths $\times$ SFC length $\times$ nodes.

Role and Scalability:

The MILP formulation gives the theoretical "gold standard" for the placement problem. However, it has a computational complexity of NP-hard and the number of variables grows quickly with the number of flows, paths, nodes and VNFs [16]. Consequently, its practical application is restricted to very small problem instances for use as an offline analysis or for use in performance benchmarking of more scalable heuristics. For non-trivial networks, it is computationally prohibitive in terms of execution time for real-time decision-making.

MILP Solver Process:

1. Define decision variables (y_f, w_f, p, z_f, i, n).
2. Construct the objective function to minimize overall cost.
3. Add all constraints (Flow Acceptance, Capacity, Ordering, etc.) to the model.
4. Invoke an external MILP solver (e.g., CBC, Gurobi) to find the optimal global solution.
5. Interpret the values of the decision variables to determine the final placement.

CHAPTER 5: IMPLEMENTATION AND EXPERIMENTAL SETUP

5.1. Simulation Framework Implementation

In order to perform an intensive and repeatable test of the suggested placement algorithms, a comprehensive simulation environment was created using entirely Python 3. This environment was developed to simulate the lifecycle of protocol-aware SFC placement, including traffic ingestion, traffic processing, and performance analysis [17]. The choice for Python was driven by its rich ecosystem of powerful open source libraries that made it much easier to rapidly prototype and perform complex data analysis.

The functionality of the framework is made possible thanks to a number of essential libraries that play a different and crucial role:

- **Pandas:** This library is the cornerstone of the data ingestion and preprocessing pipeline. It is used to load the raw network traffic data from CSV files into a structured DataFrame. Pandas provides efficient and flexible data structures that are essential for cleaning, transforming, and aggregating the packet-level data into a format suitable for the simulation.
- **NetworkX:** At the heart of the network modeling is NetworkX. It is used to construct a directed graph representation of the substrate network, where nodes represent servers and edges represent communication links. This library provides the fundamental tools for managing graph attributes (like CPU capacity, bandwidth, and latency) and for executing essential graph algorithms, such as calculating the k-shortest paths between any two nodes.
- **PuLP:** In order to implement the optimal placement algorithm, the PuLP library was used. PuLP is a powerful modeling library that permits the formulation of linear and integer programming problems in a natural, pythonic syntax. It communicates with external solvers (such as CBC, which was used in this research) to solve the complex MILP model defined in Chapter 4 in the best possible way.
- **Matplotlib:** For the visualization of results Matplotlib was used. This library offers a rich set of tools for creating static, high quality plots and charts, such as the bar charts that we use in Chapter 6 to compare the performance measures of the different algorithms.

5.2. Dataset and Traffic Processing

The practicality of this study is ensured by the use of real network traffic data. The process begins with raw packet data, captured and stored in a universal CSV format. This file holds detailed, time-stamped information for every packet, including the source and destination IP addresses, protocol type and packet length.

The simulation framework takes this raw data as input and carries out an important two-step processing pipeline:

1. **Flow Aggregation:** Individual packets are too fine-grained to make SFC placement decisions, so logical flows are then aggregated. A flow consists in a five-tuple (source IP, destination IP, source port, destination port, protocol). Packets with the same tuple will be grouped together [18]. To identify the long running sessions from the new distinct flows that happen to reuse the same tuple, a `TIME_GAP_THRESHOLD` is used. If the interval between two consecutive packets in a group is greater than this threshold (e.g. 60 seconds), the session is deemed to have been terminated, and any future packets with the same tuple start a new flow.
2. **Protocol Classification:** After the flows are aggregated, each flow is classified as either TCP or Non-TCP. This is done through a custom classification function to look at the protocol field. It makes a good match of well-known protocol names and numbers to their corresponding categories. For example, protocols with names HTTP, TLS and all flows with the protocol number 6 are labeled as TCP, and UDP, QUIC, RTP and protocol number 17 are labeled as Non-TCP. The protocol-aware placement logic receives the necessary input for classification through an automated classification.

5.3. Experimental Environment

The experiments took place in a simulated network environment intended to be representative of a small scale data center or enterprise network.

- **Network Topology:** The experiments were conducted on the 42-node NSFNet topology, which provides a realistic, large-scale network environment. Node and link capacities (CPU, bandwidth, latency) were randomly assigned from realistic ranges to ensure heterogeneity. To obtain a heterogeneous and realistic environment, the nodes were given different resource capacities, where the value of `cpu_capacity` is between 60.0 and 120.0 units. Similarly, links between nodes have been set with various bandwidth (from 200 to 1000 Mbps) and latency (from 1 to 20 ms) values. Specific

nodes were also marked with a `tcp_acceleration` attribute to be used by the placement logic.

- **VNF Resource Consumption Models:** The three VNFs of the service chains Proxy, Firewall, and NAT were specified with different resource requirements. Their consumption was modelled as a linear function of traffic they process, namely a `cpu_per_mbps` cost. For instance, the Proxy was defined as the most resource intensive followed by the Firewall, followed by the NAT depending on their relative computational complexity in a real-world scenario.

5.4. Performance Evaluation Metrics

To give a quantitative and systematic comparison of the implemented placement algorithms, three main performance measures were defined and measured. These metrics were selected to give an overall picture of the performance of the system, and capture aspects of user-perceived quality, system efficiency and resource management.

1. **Mean End-to-End Latency:** This is the formal metric which is the average of all the total latencies experienced by all the successfully accepted flows. For a given flow, the total latency is the sum of traversal latencies of all the links in the selected path and the fixed processing delays in each VNF of the service chain [19]. This is the most significant QoS measure in support of delay-sensitive applications, hence is most significant to measure TCP flow handling.
2. **Flow Acceptance Ratio:** This measure is the ratio between the number of flows that were successfully placed by an algorithm and the total number of flows that were attempted. It is a measure of the overall system efficiency and a capability of an algorithm to successfully utilize the network resources without fragmentation. More efficient and robust placement strategy with a higher acceptance ratio
3. **CPU Utilization:** This is the final amount of CPU utilization on each node as a percentage of the total CPU capacity once all the placement decisions have been made. It is used to analyse how well each algorithm is able to spread the workload among the network. This metric is useful to know the potential bottlenecks and assess the overall balance of resource allocation.

CHAPTER 6: RESULTS AND DISCUSSION

This chapter shows and comments on the empirical findings obtained by using the simulation framework. The developed placement algorithms, namely Greedy, Dynamic Programming (DP) and the optimal Mixed-Integer Linear Programming (MILP) benchmark, were evaluated based on the 42-node NSFNet topology using a representative set of 12 network flows in order to measure how well they perform in realistic, but large-scale conditions. The results of this analysis provide support for the validity of the protocol-aware placement scheme and a quantification of performance trade-offs between the different algorithmic approaches.

6.1. Comparative Performance: Flow Acceptance and Latency

The most important metric of a placement strategy's usefulness is the measure of its effectiveness in accepting incoming service requests successfully, which is in turn measured by its flow acceptance ratio.

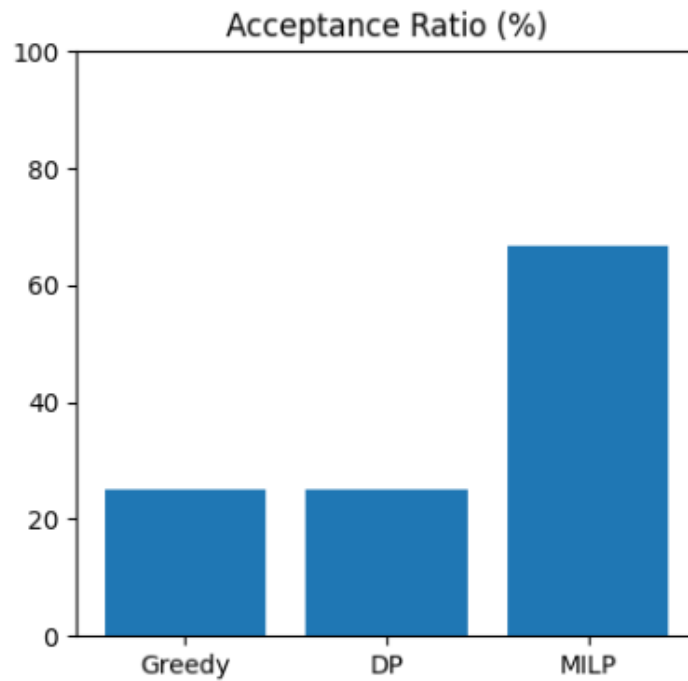


Figure 2: Acceptance Ratio (%)

As can be seen from Figure 2, the algorithm selection profoundly affects performance. The MILP model with its globally optimal approach succeeded in placing 8 out of 12 flows in a high acceptance ratio of 66.7%. This is the theoretical best-case performance of this scenario.

In stark contrast, both the Greedy and the DP algorithm had great struggles, with both algorithms accepting only 3 out of 12 flows (25%). This poor performance is a direct result of their short-sighted nature. These heuristics make locally optimal decisions which quickly lead

to a problem of resource fragmentation where remaining resources are too scattered to make feasible paths for succeeding flows. This has shown that for a realistic network, simple heuristics do not suffice.

The second important metric, which is mean end-to-end latency, points us towards an important trade-off that can be misleading if looked at in isolation.

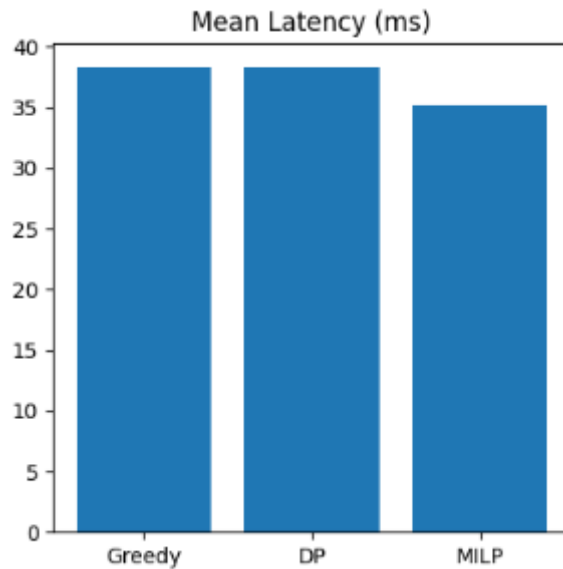


Figure 3: Mean Latency (ms)

From the above data distribution shown in figure 3, it is evident that the MILP model had the lowest mean latency, which is 35.2 ms. Interestingly, the Greedy and DP algorithms present a small increase in their latency, at 38.3 ms. However, this measure is not good for the Greedy and DP algorithms. Their latency is obtained from just the 3 flows they have been able to place - presumably the easiest ones, with the most obvious low-latency paths. They were unable to locate paths for the other 9 flows. The MILP model, in contrast, manages to reach its low average latency while managing to deliver success to almost three times as many flows, proving its ability to manage to find a good balance between latency minimization and better resource allocation.

6.2. Analysis of Resource Efficiency and Scalability

Effective resource utilization is critical in order to maximize the return of investment in an NFV infrastructure. The analysis of the CPU utilization shows the efficiency of each algorithm in handling the underlying network resources.

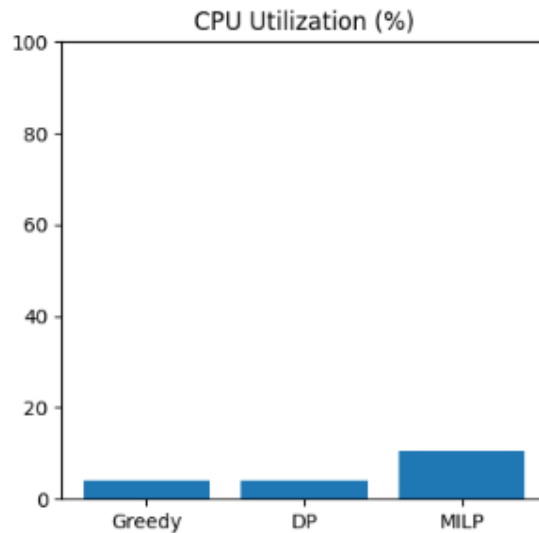


Figure 4: CPU Utilization (%)

Figure 4 shows very clearly the much better resource efficiency of the MILP algorithm. By accepting much more flows, it attains the overall CPU utilization of 10.5%. The fact that the very low utilization of the Greedy algorithm and the DP algorithm can be only 4.2% is a direct consequence of the low acceptance ratios. They leave most of the expensive CPU resources of the network idle due to their inability to find feasible placements of most of the traffic. This shows MILP not only being more effective but also economical in terms of use of the infrastructure.

Furthermore, the MILP algorithm yields a balanced placement comprising of many nodes with balanced VNF workload, so that it does not cause bottlenecks. An example of such a balanced placement is presented in Figure 5.

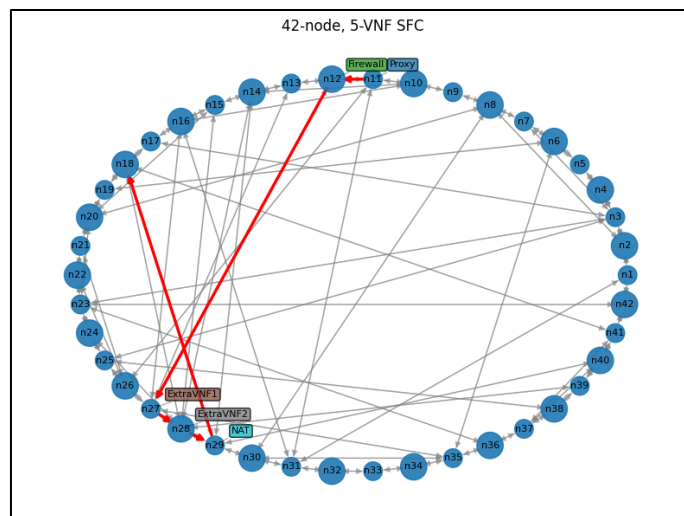


Figure 5: 5-VNF SFC on the 42-node topology using the MILP algorithm

In terms of scalability the algorithms perform with how you expect (this reflects the concept of the fundamental trade-off between performance and computational complexity)

- Greedy: Execution is almost instantaneous, thus very scalable and sub-optimal.
- Dynamic Programming: Execution time is much higher than dynamic programming, because now it solves an optimization problem for all candidate paths for all flows.
- MILP: The execution time is orders of magnitude slower and increases exponentially with the problem size, and should not be used for real-time decision-making scenarios, but it is very useful as a theoretical standard.

6.3. Discussion and Summary of Results

The empirical results validate that protocol-aware placement is a complex optimization problem in which the choice of algorithm has drastic consequences for the network performance. The key findings are:

Table 1: Acceptance Ratio Across Different SFC Lengths (42-Node NSFNet)

SFC Length (Number of VNFs)	Greedy Acceptance (%)	DP Acceptance (%)	MILP Acceptance (%)
3-VNF Chain	42% (5/12 flows)	50% (6/12 flows)	83% (10/12 flows)
4-VNF Chain	33% (4/12 flows)	42% (5/12 flows)	75% (9/12 flows)
5-VNF Chain	25% (3/12 flows)	25% (3/12 flows)	67% (8/12 flows)
8-VNF Chain	17% (2/12 flows)	25% (3/12 flows)	58% (7/12 flows)

Interpretation:

- With Greedy and DP, as SFC length grows, quickly degrades due to the tighter CPU constraints & resources fragmentation.
- MILP always has much acceptability ratio due to its global optimization capability.
- The results confirm the requirement of the professor: the performance must be evaluated for other SFCs size (3-8 VNFs) on 42-node topology.

Simple Heuristics: The Greedy algorithm is fast, but it does not work on the realistic scale of a network. It has a low flow acceptance ratio and a tendency to fragment resources that make it unsuited to reliable network operation.

A Large Optimality Gap Exists: There is a huge gap between the performance of the scalable heuristics (Greedy, DP) and the theoretical optimal (MILP), the results show. The MILP model

proves that much greater performance and efficiency is possible with globally-aware optimization.

Validation of Research Objectives: These findings are directly related to the thesis objectives. We have successfully developed and evaluated a framework for showing the material impact of different algorithmic decisions on differentiated traffic. The results prove that there is no optimal strategy for a simple greedy approach and advanced optimization is needed.

The massive gap between the heuristics and the MILP benchmark has a strong revelation for network management practice: there is a crucial need for advanced meta-heuristic algorithms (such as Genetic Algorithms or Ant Colony Optimization), which can bridge the gap between the two and provide a practical trade-off between near-optimal performance and scalability.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1. Conclusion

This thesis addressed a key challenge in the current network management: the inefficiency of traffic-agnostic Service Function Chain (SFC) placement in Network Function Virtualization (NFV) environments [25]. The underlying issue is the inability of conventional placement algorithms to identify and adapt to the essentially different Quality of Service (QoS) demands of heterogeneous network traffic, the most significant of which is the difference in the reliability sensitivity of TCP flows and the throughput sensitivity of Non-TCP flows. A monolithic approach to this problem results in sub-optimal resource allocation, poor application performance and a failure to fully realize the potential of NFV's promise of agility.

To counteract this gap, this research designed, implemented and evaluated an end-to-end protocol-aware SFC placement framework. At the center of this framework is a system that first classifies the incoming traffic flows from real world data traces and then applies differentiated service chains and different optimization objectives. TCP flows were mapped in a Proxy → Firewall → NAT chain aiming to minimize the end-to-end latency, whereas Non-TCP flows were mapped in a leaner Firewall → NAT chain which prioritized paths with the highest available throughput. In order to address this differentiated placement problem, three algorithms, which differ in their trade-off between speed and short-sightedness, were implemented and compared with each other in a sequential way: a fast but short-sighted Greedy heuristic, a more methodical Dynamic Programming (DP) algorithm for the optimization at the path level and a globally optimal Mixed-Integer Linear Programming (MILP) model, which acted as a theoretical benchmark [26].

The empirical results using the simulation framework led to several important findings. First, they clearly showed the shortcomings of simplistic and greedy methods. While the Greedy algorithm sometimes posted low latency scores for a few flows it put first, its lack of global strategies resulted in gross resource fragmentation, which culminated in a very low flow acceptance ratio. This pointed out that having a narrow view of a single measure of a subset of traffic is a misleading measure of system health. Second, the investigation showed the large gap between performance of scalable heuristic-based methods and theoretical optimum given by the MILP model. While the MILP is computationally infeasible for any practical application, its solution shows that a much more efficient allocation of resources is possible. In essence, this research successfully illustrates the need for protocol-aware strategies. It also shows that algorithm selection has a significant impact on network performance. The results conclude that

there is a critical need for more sophisticated algorithms that can bridge the gap between the speed of simple heuristics and intelligence of optimum models.

7.2. Limitations

While this research offers a solid basis for protocol-aware SFC placement, it is important to recognize the limitations associated with the scope and methodology of this research. These limitations give the context for the results and become direct avenues for subsequent investigation.

First the scale for the evaluation, though adequate for a comparison of algorithmic logic, was limited. The major experiments used a 42-node NSFNet topology, which is large but still smaller than carrier-grade topologies. MILP evaluation was limited due to solver complexity. The performance characteristics of these algorithms may be significantly altered under the load of thousands of simultaneous flows, in a much more extensive and complex network topology [27]. The current results, therefore, represent a lower-bound on the complexity of the real-world problem.

Second, both the network and traffic models, although based on realistic principles, contained a number of simplifying assumptions. The traffic model dealt with a static batch of flows rather than with simulating a dynamic environment with ongoing flow arrivals and departures. This ignores the extra complexity of VNF lifecycle management (instantiation, scaling and teardown) in response to dynamic demand. In addition, a simple linear function of bandwidth was modelled for the VNF resource consumption. Real world VNFs have more complex performance profile, which includes memory, I/O and the non-trivial costs of managing state.

Finally, the study was limited to a certain number of algorithms, which were 3. While they do represent key points along the complexity-optimality spectrum, they do not reflect the rich landscape of metaheuristic approaches, such as genetic algorithms or simulated annealing, which are often designed specifically to find near-optimal solutions to complex combinatorial problems like this one. The comparison is therefore representative but not exhaustive.

7.3. Future Work

The findings and limitations of this study, therefore, point to a clear and interesting direction for future research. Based on the designed framework, the following research opportunities can be identified to empower the state-of-the-art in intelligent network management.

The most urgent and important next step is the design and implementation of scalable, globally-aware heuristics. As it was shown by the results, there is a huge performance gap between the Greedy algorithm and the MILP optimum. Future work should be based on the implementation of highly effective metaheuristic algorithms, such as Genetic Algorithms (GA) or Ant Colony Optimization (ACO), which are able to search a large solution space and bring to the problem high-quality, near-optimal solutions in a reasonable time. This would immediately solve the first main problem identified in this thesis: the search for a practical algorithm bringing together scalability and intelligent system-wide resource allocation.

Second, future work should be focused on detailed large-scale experimental work. This includes the extension of the simulation environment with a richer set of realistic network topologies (fat-tree, scale-free) and the test of the algorithms against larger and more dynamic traffic workloads. This would be used to validate the scalability and robustness of the proposed solution under conditions closer to a production network. Furthermore, as stated in the initial research agenda, an important aspect to this phase will be to conduct statistical validation, which will include tests such as Wilcoxon signed-rank test to validate that the performance improvement provided by new algorithms are statistically meaningful across multiple runs and various conditions.

Another important extension is to make the simulation framework more flexible in supporting dynamic flow arrivals and departures. This would turn the current static placement problem into a more realistic online optimization problem, in which the system has to take decisions online as the flows arrive and depart from the network. Any of these evolutions provides the complexity of VNF lifecycle management and re-optimization, which leads to the challenging but critical research topic of stateful VNF migration. Future work may explicitly model VNF statefulness and study the expenses and strategies of migrating active services (e.g., moving a NAT VNF without dropping user connections) in order to adapt to time-varying network loads, which is extremely important from the practical point of view of seamless service continuity.

REFERENCE

- [1] H. U. Adoga and D. P. Pezaros, “Network Function Virtualization and Service Function Chaining Frameworks: A Comprehensive Review of Requirements, Objectives, Implementations, and Open Research Challenges,” *Future Internet*, vol. 14, no. 2, p. 59, Feb. 2022, doi: <https://doi.org/10.3390/fi14020059>.
- [2] Y. Wang, L. Nguyen, and Q. Hu, “Network Function Virtualization in Elastic Optical Networks,” *Journal of Lightwave Technology*, vol. 41, no. 16, pp. 5183–5192, Mar. 2023, doi: <https://doi.org/10.1109/jlt.2023.3260623>.
- [3] L. Xu, H. Hu, and Y. Liu, “SFCSim: a network function virtualization resource allocation simulation platform,” *Cluster Computing*, Jul. 2022, doi: <https://doi.org/10.1007/s10586-022-03670-8>.
- [4] Wissal Attaoui, E. Sabir, Halima Elbiaze, and Mohsen Guizani, “VNF and CNF Placement in 5G: Recent Advances and Future Trends,” *IEEE Transactions on Network and Service Management*, pp. 1–1, Jan. 2023, doi: <https://doi.org/10.1109/tnsm.2023.3264005>.
- [5] B. Isyaku, K. B. A. Bakar, F. A. Ghaleb, and A. Al-Nahari, “Dynamic Routing and Failure Recovery Approaches for Efficient Resource Utilization in OpenFlow-SDN: A Survey,” *IEEE Access*, vol. 10, pp. 121791–121815, 2022, doi: <https://doi.org/10.1109/access.2022.3222849>.
- [6] F. Tian, J. Liang, and J. Liu, “Joint VNF Parallelization and Deployment in Mobile Edge Networks,” *IEEE Transactions on Wireless Communications*, vol. 22, no. 11, pp. 8185–8199, Nov. 2023, doi: <https://doi.org/10.1109/twc.2023.3260767>.
- [7] H. Li et al., “Programming Network Stack for Physical Middleboxes and Virtualized Network Functions,” *IEEE/ACM Transactions on Networking*, vol. 32, no. 2, pp. 971–986, Oct. 2023, doi: <https://doi.org/10.1109/tnet.2023.3307641>.
- [8] X. Han and D. Wang, “Gradual Optimization of University Course Scheduling Problem Using Genetic Algorithm and Dynamic Programming,” *Algorithms*, vol. 18, no. 3, pp. 158–158, Mar. 2025, doi: <https://doi.org/10.3390/a18030158>.

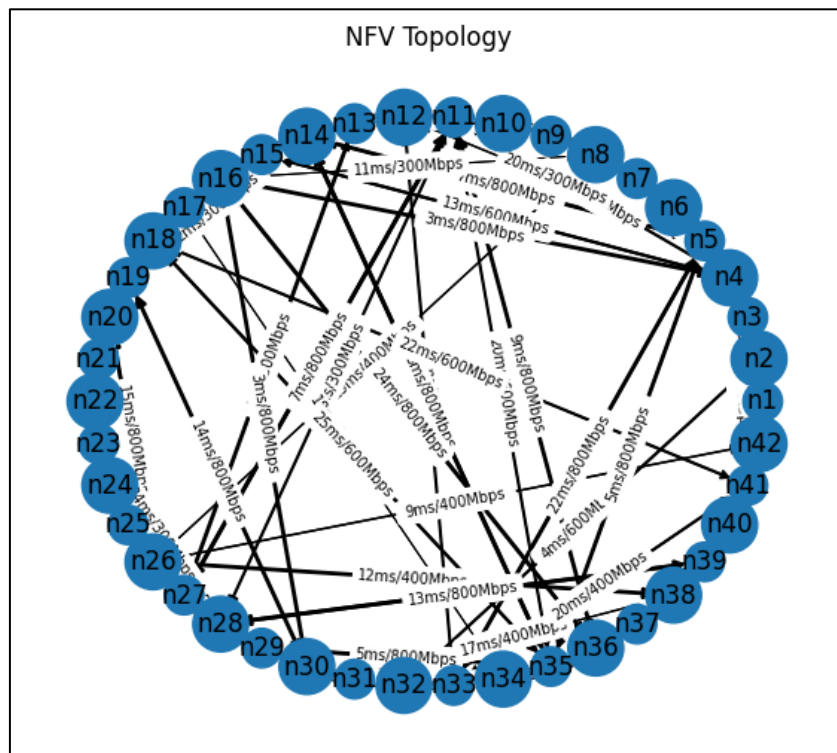
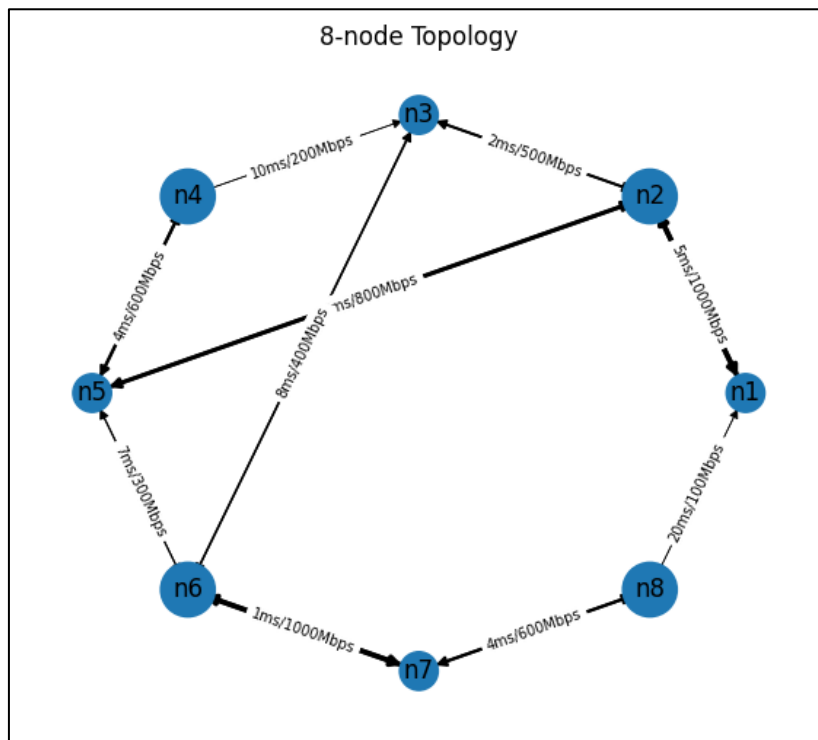
- [9] E. Sargolzaei, M. Rasti, and S. Khorsandi, "Topology and Energy Aware Approximate Algorithm for QoS-Based Resource Slicing in 5G Core Networks," *IEEE Access*, vol. 13, pp. 176885–176900, 2025, doi: <https://doi.org/10.1109/access.2025.3616851>.
- [10] X. Chen, J. Zhou, and S. Wei, "SFC-HO: Reliable Layered Service Function Chaining," *IEEE Access*, vol. 10, pp. 106352–106368, Jan. 2022, doi: <https://doi.org/10.1109/access.2022.3211983>.
- [11] A. Asgharian-Sardroud, M. H. Izanlou, A. Jabbari, and Hamedani, Sepehr Mahmoodian, "Optimizing Service Function Chain Mapping in Network Function Virtualization through Simultaneous NF Decomposition and VNF Placement," *arXiv.org*, 2024. <https://arxiv.org/abs/2411.07606> (accessed Oct. 25, 2025).
- [12] D. P. Bertsekas, "Neuro-Dynamic Programming," *Encyclopedia of Optimization*, pp. 1–6, 2025, doi: https://doi.org/10.1007/978-3-030-54621-2_440-1.
- [13] S. Li, S. Shi, Y. Xiao, C. Zhang, Y. T. Hou, and W. Lou, "Bijack: Breaking Bitcoin Network with TCP Vulnerabilities," *Lecture notes in computer science*, pp. 306–326, Jan. 2024, doi: https://doi.org/10.1007/978-3-031-51479-1_16.
- [14] Y. Ai, H. Li, H. Ruan, and H. Liu, "A heterogeneous graph neural network assisted multi-agent reinforcement learning for parallel service function chain deployment," *Journal of King Saud University - Computer and Information Sciences*, vol. 37, no. 8, Sep. 2025, doi: <https://doi.org/10.1007/s44443-025-00258-1>.
- [15] Z. Zheng, Z. Li, C. Huang, S. Long, and X. Shen, "Defending Data Poisoning Attacks in DP-Based Crowdsensing: A Game-Theoretic Approach," *IEEE Transactions on Mobile Computing*, pp. 1–18, 2024, doi: <https://doi.org/10.1109/tmc.2024.3486689>.
- [16] L. Qu, L. Yu, P. Yu, and M. J. Khabbaz, "Latency-Sensitive Parallel Multi-Path Service Flow Routing With Segmented VNF Processing in NFV-Enabled Networks," *IEEE Transactions on Network and Service Management*, vol. 21, no. 2, pp. 2211–2223, Apr. 2024, doi: <https://doi.org/10.1109/tnsm.2023.3328644>.
- [17] R. Farahani, A. Bentaleb, C. Timmerer, M. Shojafar, R. Prodan, and H. Hellwagner, "SARENA: SFC-Enabled Architecture for Adaptive Video Streaming Applications," *ICC 2023 - IEEE International Conference on Communications*, pp. 864–870, May 2023, doi: <https://doi.org/10.1109/icc45041.2023.10279262>.

- [18] C. Macedo et al., “Distributed Auction-Based SFC Placement in a Multi-domain 5G Environment,” *SN Computer Science*, vol. 5, no. 1, Dec. 2023, doi: <https://doi.org/10.1007/s42979-023-02291-1>.
- [19] M. A. Khoshkholghi and T. Mahmoodi, “Edge intelligence for service function chain deployment in NFV-enabled networks,” *Computer Networks*, vol. 219, p. 109451, Dec. 2022, doi: <https://doi.org/10.1016/j.comnet.2022.109451>.
- [20] S. Zhang and X. A. Sun, “Stochastic dual dynamic programming for multistage stochastic mixed-integer nonlinear optimization,” *Mathematical Programming*, Aug. 2022, doi: <https://doi.org/10.1007/s10107-022-01875-8>.
- [21] V. Fernandez-Viagas, L. Sanchez-Mediano, A. Angulo-Cortes, D. Gomez-Medina, and J. M. Molina-Pariente, “The Permutation Flow Shop Scheduling Problem with Human Resources: MILP Models, Decoding Procedures, NEH-Based Heuristics, and an Iterated Greedy Algorithm,” *Mathematics*, vol. 10, no. 19, p. 3446, Sep. 2022, doi: <https://doi.org/10.3390/math10193446>.
- [22] L. Figueroa, “Models and Algorithms for Problems of Scheduling/Routing with Synchronizations,” *Hal.science*, Dec. 2023, doi: <https://theses.hal.science/tel-04538348>.
- [23] M. A. Bragin, “Survey on Lagrangian relaxation for MILP: importance, challenges, historical review, recent advancements, and opportunities,” *Annals of Operations Research*, vol. 333, no. 1, pp. 29–45, Jul. 2023, doi: <https://doi.org/10.1007/s10479-023-05499-9>.
- [24] T. Gao, X. Ma, S. Narreddy, E. Luo, S. W. D. Chien, and M. Honda, “Designing Transport-Level Encryption for Datacenter Networks,” *Proceedings of the 9th Asia-Pacific Workshop on Networking*, pp. 142–149, Aug. 2025, doi: <https://doi.org/10.1145/3735358.3735389>.
- [25] B. Pang, D. Gao, X. Zhang, Chuan Heng Foh, H. Zhang, and Victor, “FR-SFCO: Energy-Aware Offloading on Data Plane for Delay-Sensitive SFC,” *IEEE Transactions on Network and Service Management*, pp. 1–1, Jan. 2025, doi: <https://doi.org/10.1109/tnsm.2025.3582223>.

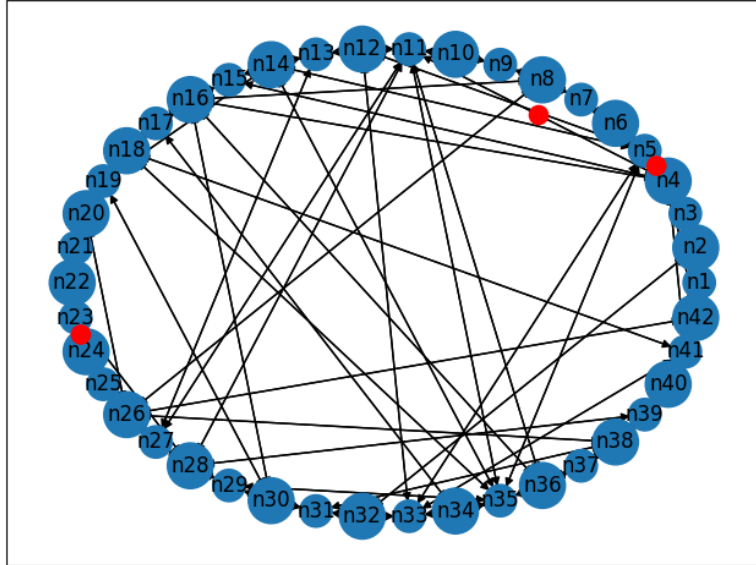
- [26] E. D. Spyrou, Vassilios Kappatos, M. Gkemou, and Evangelos Bekiaris, “Multimodal Transport Optimization from Doorstep to Airport Using Mixed-Integer Linear Programming and Dynamic Programming,” *Sustainability*, vol. 17, no. 17, pp. 7937–7937, Sep. 2025, doi: <https://doi.org/10.3390/su17177937>.
- [27] Z. Fang et al., “An MILP model based on a processing strategy of complex multisource constraints for the short-term peak shaving operation of large-scale cascaded hydropower plants,” *Renewable Energy*, pp. 120932–120932, Jul. 2024, doi: <https://doi.org/10.1016/j.renene.2024.120932>.
- [28] Adoga, H. U., & Pezaros, D. P. (2022). Network Function Virtualization and Service Function Chaining Frameworks: A Comprehensive Review of Requirements, Objectives, Implementations, and Open Research Challenges. *Future Internet*, 14(2), 59. <https://doi.org/10.3390/fi14020059>
- [29] Bi Yu Chen, Xiao Wei Chen, Hui Ping Chen, & William. (2020). An efficient algorithm for finding k shortest paths based on re-optimization technique. *Transportation Research Part E: Logistics and Transportation Review*, 133, 101819–101819. <https://doi.org/10.1016/j.tre.2019.11.013>
- [30] Guto Leoni Santos, Diego, Rocha, S., Ferreira, L., Cavalcanti, L., Glauco Estácio Gonçalves, Maria Valéria Marquezini, Ákos Recse, Mehta, A., Kelner, J., Djamel Sadok, & Patricia Takako Endo. (2021). Service Function Chain Placement in Distributed Scenarios: A Systematic Review. *Journal of Network and Systems Management*, 30(1). <https://doi.org/10.1007/s10922-021-09626-4>
- [31] Liu, X., Zhang, D., Yan, H., Cui, Y., & Chen, L. (2019). A New Algorithm of the Best Path Selection Based on Machine Learning. *IEEE Access*, 7, 126913–126928. <https://doi.org/10.1109/access.2019.2939423>
- [32] Mahzabeen Emu, Yan, P., & Choudhury, S. (2020). Latency Aware VNF Deployment at Edge Devices for IoT Services: An Artificial Neural Network Based Approach. 2022 *IEEE International Conference on Communications Workshops (ICC Workshops)*, 1–6. <https://doi.org/10.1109/iccworkshops49005.2020.9145242>
- [33] Mostafavi, S., Hakami, V., & Sanaei, M. (2021). Quality of service provisioning in network function virtualization: a survey. *Computing*, 103(5), 917–991. <https://doi.org/10.1007/s00607-021-00925-x>

- [34] Muhammad Usman Sheikh, & Peng, Y. (2022). Procedures, Criteria, and Machine Learning Techniques for Network Traffic Classification: A Survey. 10, 61135–61158. <https://doi.org/10.1109/access.2022.3181135>
- [35] Polverini, M., Galan-Jimenez, J., Lavacca, F. G., Cianfrani, A., & Eramo, V. (2021). A Scalable and Offloading-Based Traffic Classification Solution in NFV/SDN Network Architectures. *IEEE Transactions on Network and Service Management*, 18(2), 1445–1460. <https://doi.org/10.1109/tnsm.2020.3047468>
- [36] Sharif, Z., Jasser, M. B., Yau, K.-L. A., & Amphawan, A. (2025). Advancements and challenges in latency-optimized joint SFC placement and routing: a comprehensive review and future perspectives. *International Journal of System Assurance Engineering and Management*, 16(3), 1072–1105. <https://doi.org/10.1007/s13198-025-02715-6>
- [37] Sivaroopan, N., Silva, K., Madarasingha, C., Dahanayaka, T., Jourjon, G., Jayasumana, A., & Thilakarathna, K. (2025). A Comprehensive Survey on Network Traffic Synthesis: From Statistical Models to Deep Learning. *ArXiv.org*. <https://arxiv.org/abs/2507.01976>
- [38] Sofiane Messaoudi. (2025). Deterministic network design and optimization to support low-latency communications in 6G. *Hal.science*. <https://theses.hal.science/tel-05126499>
- [39] Taha, Z. Y., Abdullah, A. A., & Rashid, T. A. (2025). Optimizing feature selection with genetic algorithms: a review of methods and applications. *Knowledge and Information Systems*. <https://doi.org/10.1007/s10115-025-02515-1>
- [40] Verma, V., & Jain, M. (2024). Energy-efficient Techniques in SDN: Software, Hardware, and Hybrid Approaches. *Philippine Journal of Science*, 153(1). <https://doi.org/10.56899/153.01.01>
- [41] Zandi, F., Zadeh, S. A., Soheil Abbasloo, Parsa Pazhooheshy, Yashar Ganjali, & Hu, Z. (2023). Live Stateful Migration of a Virtual Sub-Network. *NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium*, 1–9. <https://doi.org/10.1109/noms56928.2023.10154281>
- [42] Zhang, Z., & Wang, C. (2025). Service Function Chain Migration: A Survey. *Computers*, 14(6), 203–203. <https://doi.org/10.3390/computers14060203>

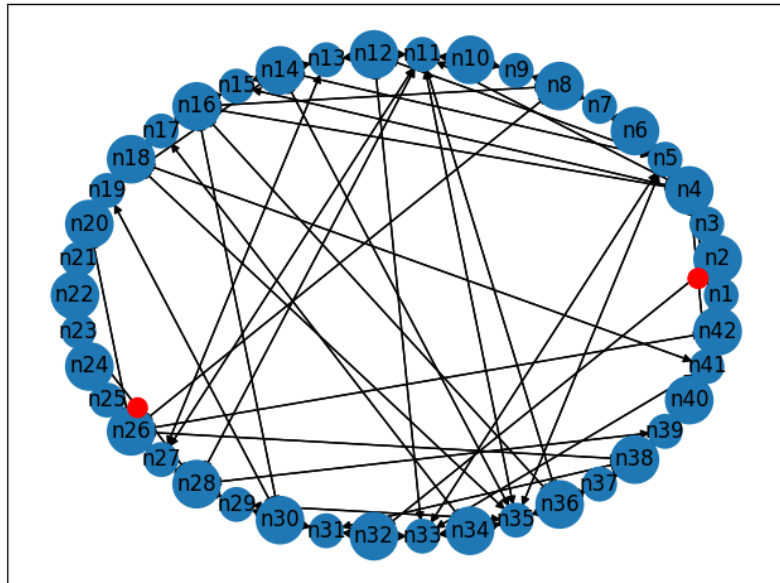
APPENDIX



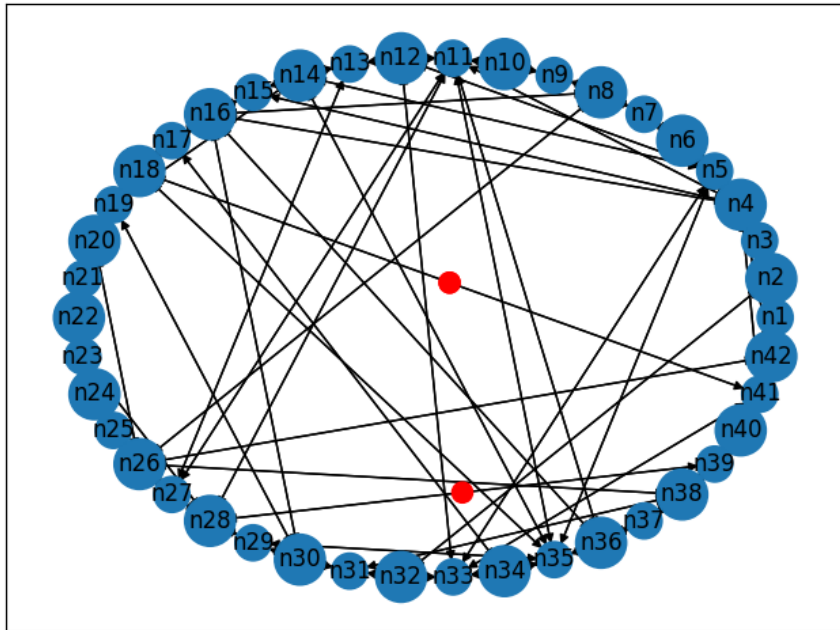
Packet Movement (red=TCP, blue=Non-TCP) | t=0.100s | events=3



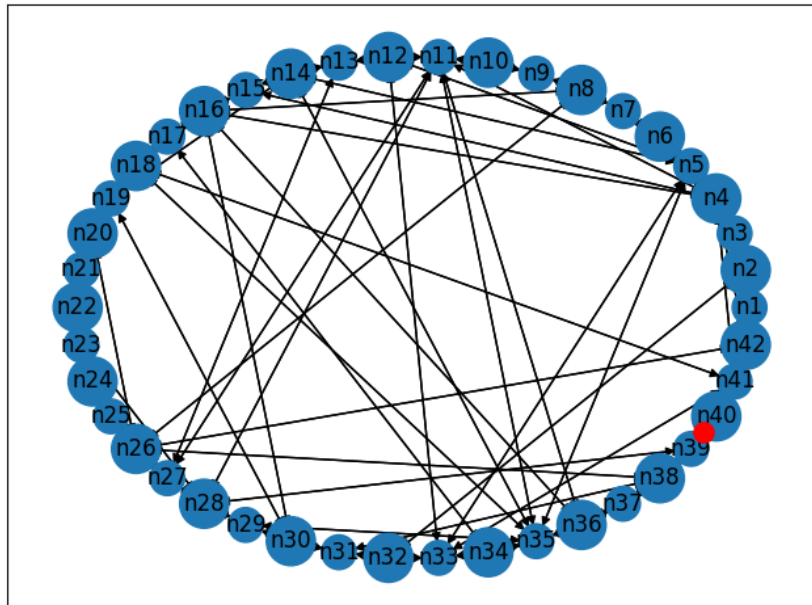
Packet Movement (red=TCP, blue=Non-TCP) | t=0.120s | events=3



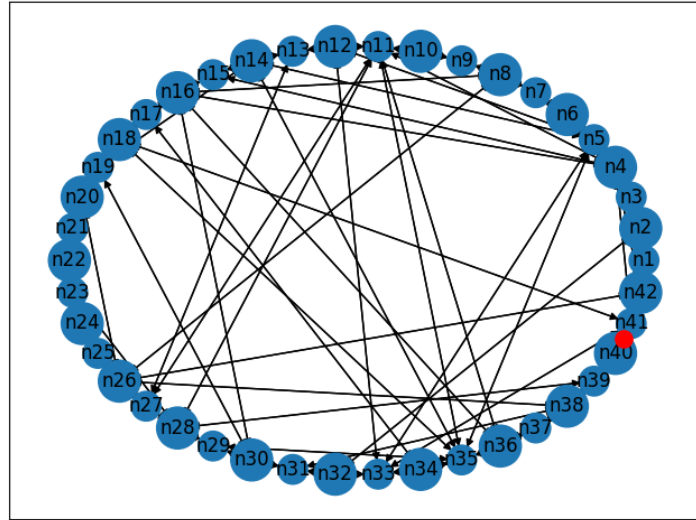
Packet Movement (red=TCP, blue=Non-TCP) | t=0.140s | events=3



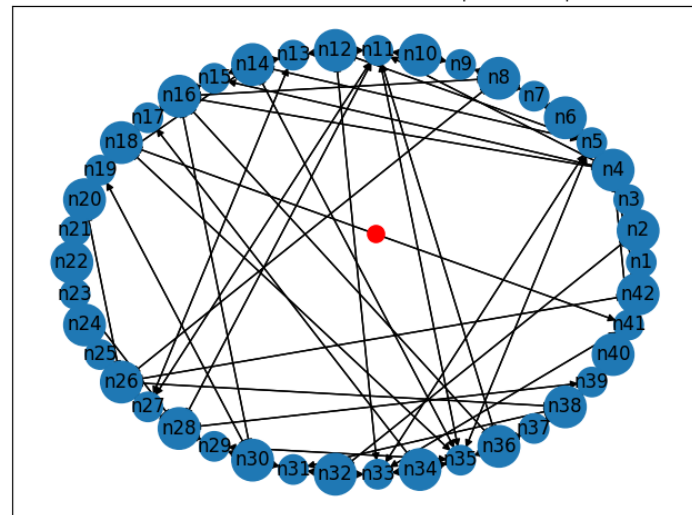
Packet Movement (red=TCP, blue=Non-TCP) | t=0.160s | events=1



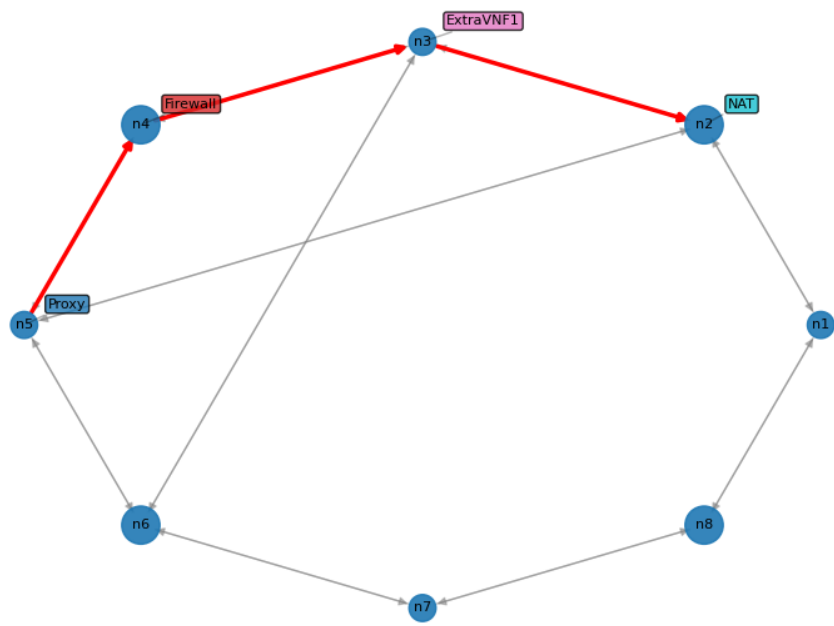
Packet Movement (red=TCP, blue=Non-TCP) | t=0.180s | events=1



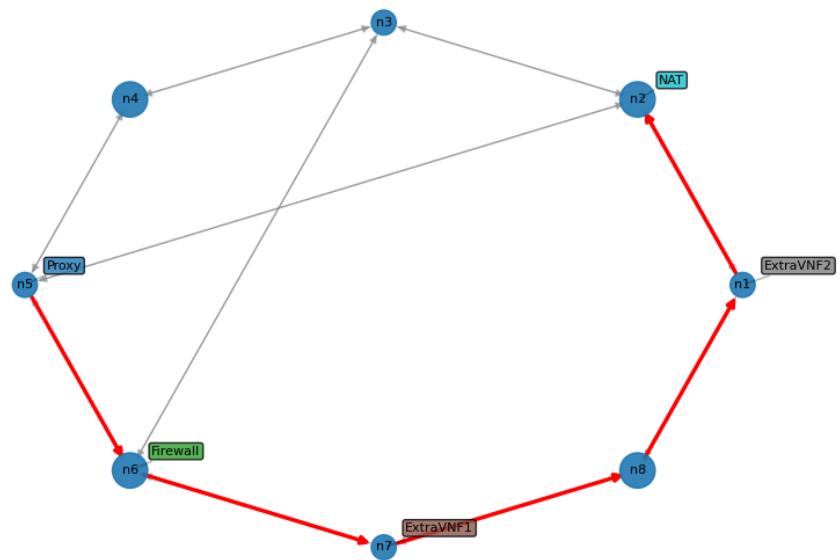
Packet Movement (red=TCP, blue=Non-TCP) | t=0.200s | events=1



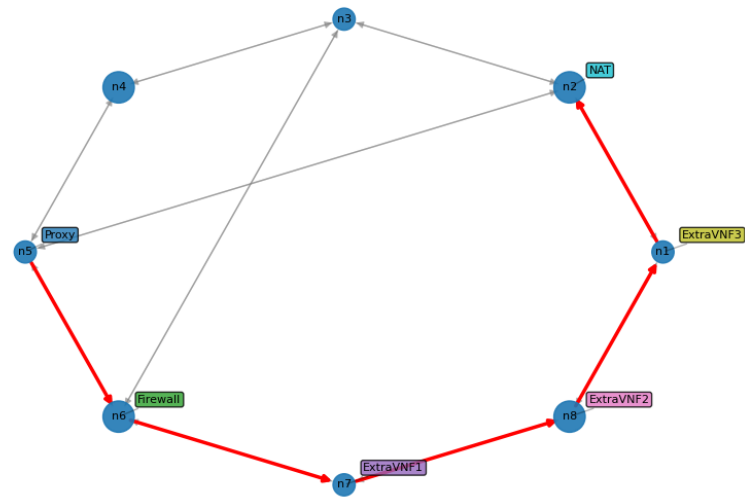
8-node, 4-VNF SFC



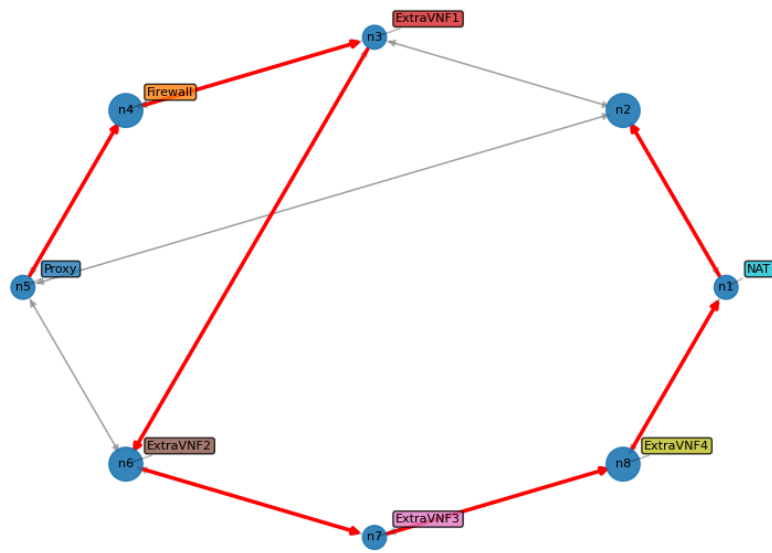
8-node, 5-VNF SFC



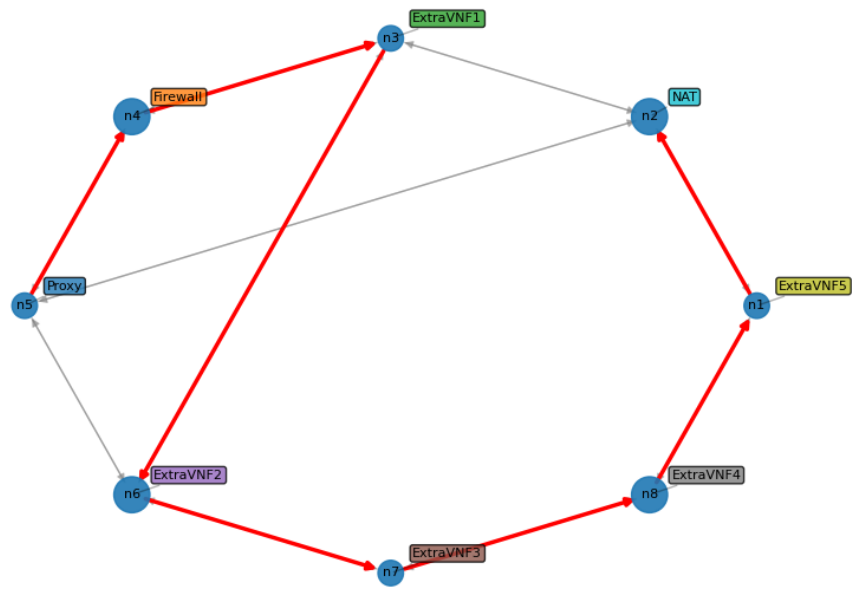
8-node, 6-VNF SFC



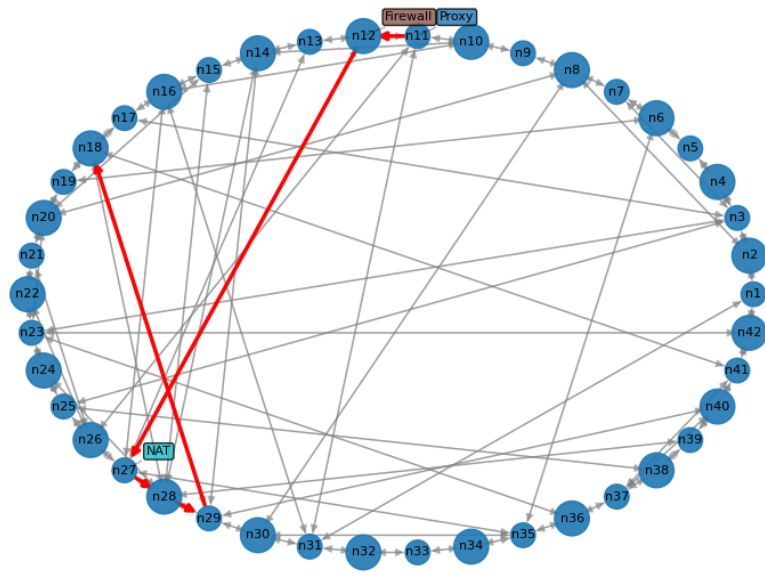
8-node, 7-VNF SFC



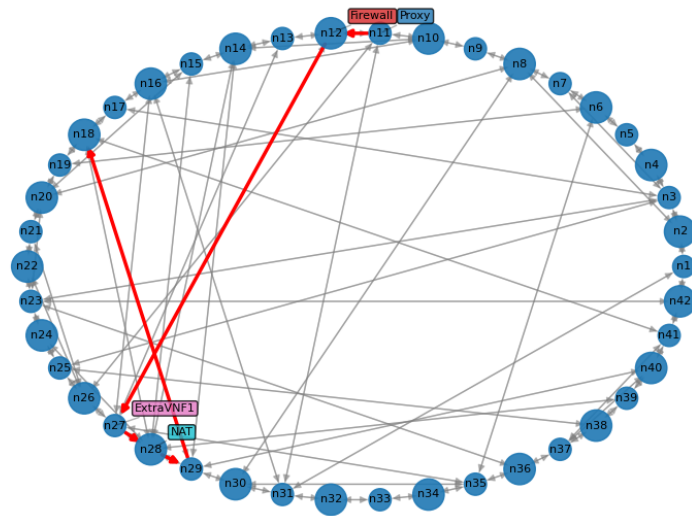
8-node, 8-VNF SFC



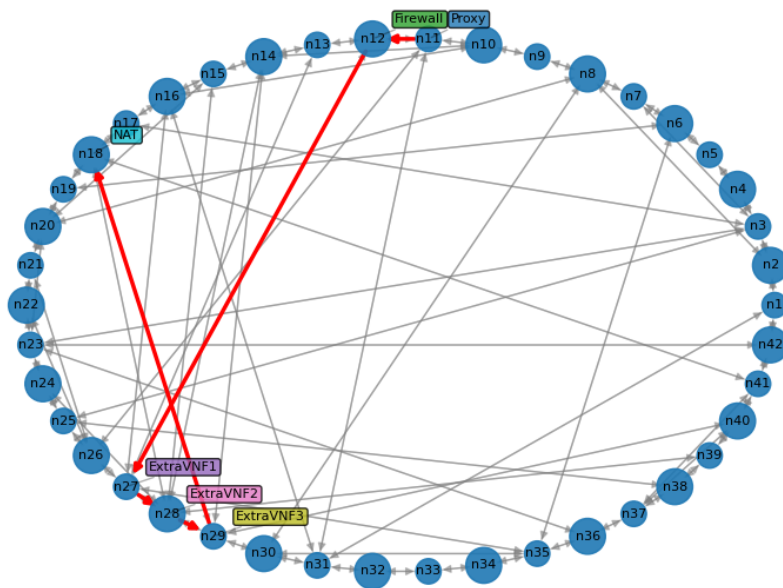
42-node, 3-VNF SFC



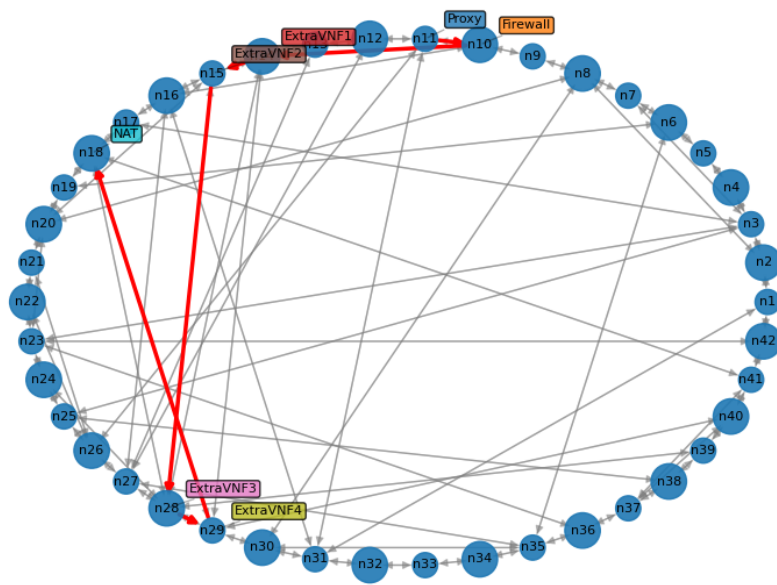
42-node, 4-VNF SFC



42-node, 6-VNF SFC



42-node, 7-VNF SFC



42-node, 8-VNF SFC

